SilentProof: Anonymous Authentication with Blockchain-Backed Offloading

Jamal H. Mosakheil
University of Memphis
Memphis, USA
jmskheil@memphis.edu

Kan Yang
University of Memphis
Memphis, USA
kan.yang@memphis.edu

ABSTRACT

Anonymous credentials empower users with strong privacy controls by enabling them to selectively disclose specific attributes from a credential without unveiling other personal details. However, current anonymous credential systems confront two primary challenges: 1) Their designs are not tailored for resource-limited devices due to their dependence on computationally heavy techniques such as generic zero-knowledge or Groth-Sahai proofs at both presentation and verification phases; and 2) They pose complexities in deploying anonymous credentials as-a-service since users must maintain specific key materials and dedicated software. To tackle these challenges, we present SilentProof, a blockchain-backed anonymous authentication protocol, which offloads the responsibility of credential presentation from individual users to the blockchain. By harnessing the combined strengths of zero-knowledge proxy re-identification and blind signatures, SilentProof empowers the blockchain to create zero-knowledge proofs for users, while ensuring clear user consent and eliminating the need for direct access to user-specific attributes. This design broadens the use cases for anonymous credentials, making it ideal for scalable, secure, and privacy-centric authentication, especially for resource-limited clients. Furthermore, SilentProof lays the groundwork for a new era of privacy-preserving identity management as-a-service. We provide rigorous security proofs to validate our approach and have developed a SilentProof prototype, showcasing the protocol's practical efficiency.

CCS CONCEPTS

• **Security and privacy → Pseudonymity, anonymity and untraceability**; **Authentication**.

KEYWORDS

SilentProof, Blockchain-backed Anonymous Authentication, Anonymous Credentials as-a-service, Resource-constrained Devices

ACM Reference Format:
Jamal H. Mosakheil and Kan Yang. 2024. SilentProof: Anonymous Authentication with Blockchain-Backed Offloading. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '24), July 1–5, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3634737.3637635

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASIA CCS '24, July 1–5, 2024, Singapore, Singapore
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0482-6/24/07…$15.00
https://doi.org/10.1145/3634737.3637635

1 INTRODUCTION

Anonymous credentials (ACs) are cryptographic mechanisms that enable user-centered identity management, offering recipients robust authentication and user privacy. These credentials are issued by certifying authorities, validating specific user attributes encoded within them. Users can use these credentials to reveal select attributes or verify conditions (e.g., being of legal drinking age) without unveiling their identity or exact attribute details. These credentials encompass a range of attributes, from basic ones like age or address for individuals to complex configurations for platforms, crucial for tasks like remote attestation [19]. Moreover, the system guarantees that repeated credential presentations cannot be linked, enhancing user anonymity and preventing potential tracking of user actions. While research on ACs systems has advanced robust authentication and user privacy, it has largely overlooked modern use cases, limiting its broad-scale deployment for many clients and users. Existing ACs schemes, due to their cryptographic foundation, face two primary obstacles: they are not tailored for resource-constrained devices and pose difficulties in adopting ACs as-a-service deployments.

Most of the existing general-purpose anonymous credential schemes [14, 17, 18, 41] in the literature typically involve two constructions: *credential issuance* and *credential presentation*. Credential issuance is used to receive certified credentials, while credential presentation allows for the selective disclosure of attributes encoded within the credentials. However, existing anonymous credential schemes primarily focus on the issuance process and utilize computationally expensive generic zero-knowledge proofs or Groth-Sahai proofs for credential presentation and verification. This approach leads to inefficient solutions, limiting scalability for modern use cases involving resource-constrained devices, IoT devices, sensors, wearables, smartcards, RFIDs, and even smartphones with limited computational power and battery life.

Furthermore, while ACs grant users the liberty to determine which attributes they disclose, the current ACs schemes require access to all attributes in the clear (unencrypted) when proving possession of a credential to a third party. This poses a challenge for offering as-a-service identity management systems. Cloud or distributed ledger platforms like blockchain can be ideal infrastructure for deploying ACs as as-a-service and scaling it to millions or billions of users and clients. However, implementing the existing ACs scheme on Cloud or blockchain platforms would directly undermine the initial motivation of ACs: protecting user privacy and

control over attribute disclosure. To mitigate these concerns, users would need key material and perform local computations, whether partial or complete, to avoid exposing attribute values to the cloud provider or blockchain [31]. Nevertheless, this local computation approach might be inadequate for resource-limited devices such as sensors or smart cards intended to authenticate users or devices to a service. For example, performing resource-intensive operations like pairing or full-length exponentiations, possibly within RSA groups, could be infeasible on these devices.

To address these issues, we present SilentProof, a blockchain-backed anonymous authentication protocol, focusing on enhancing the *credential presentation* aspect of anonymous credentials. SilentProof delegates the responsibility of presenting and verifying user anonymous credentials to the blockchain. This enables the blockchain to generate zero-knowledge proofs on behalf of the user, showcasing credential knowledge to verifiers. Importantly, this process occurs without the blockchain accessing the user's actual attribute values. Instead, the user's consent is required to allow representation to a service provider. This consent mechanism prevents unauthorized sharing of user credentials [30], addressing issues observed in ACs commercial products like U-Prove [36] or Idemix [15]. In this delegated presentation, the user only computes one group element (the consent token), while the remaining computation is carried out by the blockchain.

The primary goal of SilentProof design is to expand the conventional credential presentation in existing ACs schemes to a delegable credential presentation while ensuring coherence with existing general-purpose ACs schemes [37, 41] for *credential issuance*. Instead of creating an entirely new credential issuance scheme, SilentProof is designed to work in conjunction with established general-purpose ACs schemes [37, 41] used for issuing credentials. As a result, users can acquire anonymous credentials initially through existing schemes in traditional single issuer setups [37] or distributed and threshold issuance [41] through blockchain smart contracts. Once they have obtained these credentials, they can use the SilentProof for presenting and verifying their credentials whenever necessary for a service. This transformation in ACs' presentation structure not only facilitates the use of ACs for resource-limited devices but also enables their deployment as-a-service to effectively scale for modern use cases. Despite this transformation, users or clients still retain control over their attributes and selective disclosure, thus upholding the original goal of ACs.

The SilentProof comprises two schemes: Credential Outsourcing (CO) and Delegable Credential Presentation (DCP). In CO, users receive credentials for both public (e.g., name, public key) and private attributes (e.g., age, private key) from certifying authorities. They then register with the blockchain to request delegation permission and securely outsource their credentials and encrypted attributes to the blockchain as a one-time setup. DCP empowers the blockchain to present users' credentials to service providers. It generates zero-knowledge proofs on behalf of users, selectively disclosing or proving possession of attributes for each service request. SilentProof employs short and computationally efficient credentials and consists of only two group elements, irrespective of the number of attributes encoded in a credential or issuing authorities. In our security model, the blockchain acts as a semi-trusted intermediary, not colluding with users, service providers (verifier), or credential issuers, and cannot independently act as a user or verifier.

It is important to note that the SilentProof protocol could be deployed on a Cloud infrastructure instead of a blockchain. However, its primary objective is to integrate with blockchain to expand the use of ACs beyond traditional cases, like providing identification [22], access control to online-service [38], and anonymous subscriptions [32] through their e-ID, vaccination card, or e-tickets [34]. For instance, the SilentProof protocol could be implemented as a smart contract that attests some claims about a user operating through the contract, such as their identity, attributes, and their wallet balance. This could involve activities like participating in privacy-preserving petitions within decentralized autonomous organizations (DAOs) or proving asset ownership on DeFi platforms without transmitting the assets, thereby enhancing privacy and security. Moreover, implementing SilentProof as a smart contract on established blockchains such as Ethereum or other layer-1/layer-2 solutions substantially cuts infrastructure costs. We assume that security and privacy risks remain within the confines of our threat model. This approach guarantees confidentiality, authenticity, and availability, enabling scalable, on-demand presentation and verification of anonymous credentials for a range of envisioned applications. Table 1 compares SilentProof with other relevant cryptographic constructions.

The contributions of this paper are summarized as follows.

- We propose SilentProof, a blockchain-backed anonymous authentication protocol, which offloads the responsibility of credential presentation from individual users to the blockchain while ensuring user consent and eliminating the need for direct access to user-specific attributes.
- By leveraging zero-knowledge proxy re-identification [11] and blind signatures [37], we present a concrete construction of SilentProof consisting of a Credential Outsourcing scheme and a Delegable Credential Presentation scheme. These two schemes empower the blockchain to create zero-knowledge proofs on behalf of users under their consent.
- We prove that SilentProof is complete, sound and honest verifier zero-knowledge. Moreover, we implement a prototype of SilentProof to demonstrate its functionality, efficiency, and robustness.

The rest of the paper is organized as follows: Section 2 provides an overview of the related work. Section 3 defines the problem statement, including system and threat models and design goals. Section 4 discusses the building blocks. Section 5 presents SilentProof overview and algorithms designs. In Section 6, the concrete construction of SilentProof schemes and implementation of multi-attribute SilentProof are discussed. Security analysis is discussed in Section 7, and performance evaluation is presented in Section 8. Finally, Section 9 concludes the paper. The appendixes contain building blocks, correctness proof, comprehensive security theorem proofs, and detailed algorithms for multi-attribute SilentProof.

2 RELATED WORK

Since the inception of anonymous credentials by Chaum [18], numerous research efforts have focused on developing anonymous credential protocols and implementations [3, 4, 11, 13, 14, 26, 37, 39, 41].

Table 1: Comparing SilentProof with relevant constructions (Signature size is evaluated asymptotically or based on the number of group elements. Here, q represents the count of attributes embedded in the credentials.)

Scheme	delegation	as-a-service	general-purpose	user-consent	multi-show unlinkable	credential-size
[31] Kren et al.	✓	✓	✓	✗	✓	$\mathcal{O}(q)$ Elements
[29] Haböck et al.	✓	✓	✓	✓	✓	$\mathcal{O}(q)$ Elements
[37] Pointcheval and Sanders	✗	✗	✓	N/A	✓	2 Elements
[41] Sonnino et al.	✗	✗	✓	N/A	✓	2 Elements
[15] Idemix	✗	✗	✓	N/A	✗	$\mathcal{O}(q)$ Elements
[23] U-Prove	✗	✗	✓	N/A	✗	$\mathcal{O}(q)$ Elements
[12] Camenisch et al.	partial (SIM → phone)	✗	✗	✗	✓	4 Elements
[30] Hanzlik and Daniel	partial (SIM → phone)	✗	✓	✗	✓	$\mathcal{O}(q)$ Elements
Our (SilentProof)	✓	✓	✓	✓	✓	2 Elements

Notable examples include Microsoft's U-Prove based on Brands signatures [10, 35] and IBM's identity mixer on CL-signatures [14, 15]. However, a common limitation of these approaches is that presenting a credential necessitates accessing the user's private attribute value. This limits the use of anonymous credentials to offer as-a-service for cloud or blockchain identity services.

However, our comparison focuses exclusively on anonymous credential protocols that support an outsourced credential presentation [20, 24, 29, 31]. Krenn et al. [31] proposed an encrypted attribute-based credentials system where the majority of credential presentation computations are outsourced to a centralized wallet in the cloud. In their approach, users obtain credentials encrypted under their public keys and upload them, along with a re-encryption key, to a cloud-based centralized wallet. To prove possession, the wallet re-encrypts the ciphertexts to the public key of the intended third party and provides a zero-knowledge proof of the correct execution of the necessary computations.

Krenn et al. [31] addressed limitations in efficient credential presentation on the user's side, enabling the provision of anonymous credentials as a service. However, a recent attack [29] on their scheme revealed vulnerabilities in the bi-directional multi-hop proxy re-encryption, allowing a malicious proxy to recover secret keys. Haböck [29] proposed a revised scheme, requiring additional exponentiation during credential proving to mitigate the attack. Both Krenn et al. [31] and Haböck [29] schemes use zero-knowledge proof to verify re-encryption correctness instead of proving attributes directly. This limitation restricts their use when proving predicates on attributes. Moreover, employing structure-preserving signatures results in a linear growth of credential size with the number of attributes, leading to relatively large credential and proof sizes. Fuchsbauer et al. [2] propose a novel approach using a structure-preserving signature (SPS) scheme on equivalence classes (SPS-EQ) and set commitments. This approach significantly reduces the user's computational effort, requiring only five exponentiations on an elliptic curve, the number of undisclosed attributes, and a two-exponent zero-knowledge proof. However, in their work, the user is responsible for directly providing proof of possession of their attributes without the involvement of a third-party delegation. This limitation prevents the scheme from being used as an anonymous credential as-a-service solution.

Direct Anonymous Attestation with Attributes (DAA-A) [20] employs a distinct approach, utilizing a Trusted Platform Module (TPM) for attestation. In DAA-A, a signed authentication token is presented to the verifier, with the TPM chip performing a minor computation while the local host processor handles the pairing calculation. However, DAA-A significantly differs from anonymous credentials regarding trust assumptions on the host, which are unrealistic and undesirable when considering the provision of anonymous credentials as-a-service. Contrastingly, our proposed solution operates autonomously, without relying on the TPM or trusted execution environments like Intel SGX [21]or TDX [1].

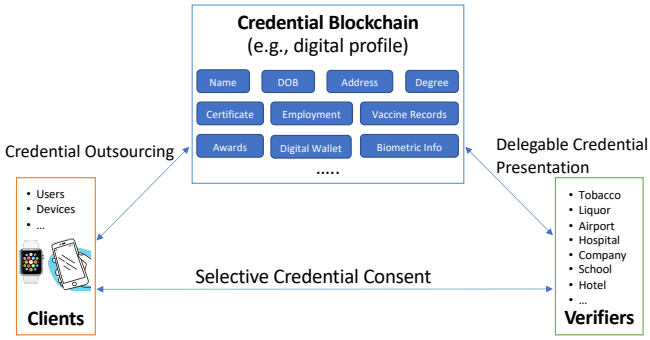

Figure 1: System Model

3 PROBLEM STATEMENT

3.1 System Model

As depicted in Figure 1, our system includes clients, a credential blockchain, and verifiers.

Clients. The term "clients" refers to a diverse array of entities, encompassing users, devices, and organizations. In this context, a client's role involves acquiring anonymous credentials from various issuers, which are then securely stored within the credential blockchain. All the accumulated credentials form a comprehensive digital profile within the blockchain. Notably, clients are often resource-constrained, typified by their reliance on devices like smartphones or smartwatches. Given this constraint, it is impractical for clients to download and exhibit each individual credential when faced with authentication requests. Instead, clients seek to offload the associated computational tasks to the credential blockchain. Consequently, blockchain assumes the responsibility of furnishing verifiers with proof regarding the existence of these anonymous credentials, all while upholding the client's consent.

Credential Blockchain (delegator). At the core of this architecture is the credential blockchain, entrusted with managing and storing all anonymous credentials tied to the clients. The process of adding a credential to a client's profile necessitates the blockchain's

verification of the credential's authenticity and its rightful association with the client. Crucially, this validation is accomplished without the blockchain ever accessing the actual content of the credential. In the context of authentication, the blockchain generates proofs on behalf of the clients, abiding by their consent.

Verifiers. Verifiers encompass a broad spectrum of entities, spanning individuals and systems alike, all necessitating the authentication of clients' credentials. Examples include tobacco or liquor stores, airport check-in counters, hospitals, schools, and hotels. The verification process transpires through a communication channel that involves the client's mobile device. Diverse technologies like QR codes, Near Field Communication, Bluetooth, WiFi, or RFID facilitate this communication, allowing verifiers to solicit a specific set of credentials from clients.

3.2 Threat Model

The threat model is defined as follows:

- **Compromised Blockchain node (delegator).** A compromised blockchain validator or node may execute the following attacks: 1) Information Disclosure: Reveals the user's secret information, such as the user's private key from the validator re-proof key rk, user private attribute value from attribute commitment, or from the user's initial proof. 2) User Impersonation: Attempting to impersonate a user by obtaining the user's private attributes encoded in anonymous credentials or by creating fake credentials. 3) Delegation Tampering: Illegitimate actions in the delegation process, gaining unauthorized delegation rights or transferring existing rights to another delegator. 4) Unauthorized Proof Generation: Generating a user attribute possession proof without the user's consent. 5) Collusion with Verifier: Collaborating with a dishonest verifier to expose the user's private key from the re-proof or attribute verification key or token (Section D.4). 6) Replay Attack: Reusing a previous user's attribute token (tk) for illegitimate service access.
- **Compromised verifier.** A compromised verifier may execute the following attacks: 1) Traceability Attack: Linking a user's anonymous credentials to their real-world identity through correlation attacks or compromising the anonymity-preserving mechanisms of the protocol. 2) Information Disclosure: Revealing the user or validator's sensitive information, such as user and delegator private keys, user attributes' private values, and credentials. 3) Replay Attack: Trying to execute a replay attack by reusing previous proofs.
- **Compromised client.** Unauthorized Access: A compromised client, such as a user or any external adversary, may attempt to request a delegator to generate proof on their behalf to gain illegitimate access from the service provider.

3.3 Design Goals

To counteract the attacks outlined in the threat model and enable the offloading of authentication responsibilities, SilentProof must accomplish the following objectives:

- **Delegable Presentation.** SilentProof should facilitate the delegation of the authentication process to a semi-trusted third party, such as a blockchain, which requires generating zero-knowledge proof for possession of user attributes and presenting user credentials to a service provider without compromising the user's privacy or learning about the user attribute's private value.
- **Selective disclosure.** To preserve user privacy, SilentProof must ensure that the blockchain only discloses the selected attributes to the designated service provider.
- **Unlinkability**: SilentProof supports unlinkable multi-show credentials, ensuring user identity remains unlinked across multiple presentations of credentials.
- **User Consent.** SilentProof includes a mechanism that ensures user consent is obtained before the blockchain generates a zero-knowledge proof on the user's behalf. The user also has the ability to select the attribute to be disclosed.
- **Anonymous Credential as-a-service.** The SilentProof design facilitates offering users anonymous credentials with selective disclosure as-a-service by a semi-trusted third party like blockchain.
- **Resource-efficiency.** SilentProof should be designed to minimize resource usage on the resource-constraint devices by outsourcing resource-intensive computations and storage to a semi-trusted third party.
- **Scalability.** SilentProof should be able to handle a large number of users and devices and be able to handle a high number of authentication requests.
- **Security.** SilentProof should be robust against collusion, impersonation, replay, and Sybil attacks.

4 PRELIMINARIES

4.1 Pointcheval and Sanders (PS) Credentials

Below, we present the anonymous credentials scheme proposed by Pointcheval and Sanders (PS) [37], which provides blindness, unlinkability, efficiency, and short credentials. We adopted the PS signature scheme as the foundational *credential issuance* protocol for our SilentProof. Our SilentProof framework is also adaptable to Coconut credentials scheme [41], which means anonymous credentials issued using Coconut can be delegable presented by SilentProof. The Coconut scheme is also based on the PS signature. However, Coconut employs a decentralized approach, where a threshold number of authorities issue anonymous credentials through smart contracts.

- **PS.Setup**$(1^\lambda) \rightarrow (param)$: It chooses a bilinear group $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ with order q, where q is a λ-bit prime number as input. Let g_1 be a generator of G_1 and g_2 a generator of G_2. The system parameters are $params = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, g_1, g_2)$.
- **PS.KeyGen**$(params) \rightarrow (sk, pk)$: The system parameters $params$ are taken as input, and g_2 is randomly selected from $\mathbb{G}_2^*$. The secret key sk is randomly chosen as $(x, y) \in \mathbb{Z}_q^*$, and then (X, Y) is computed as (g_2^x, g_2^y). The public key pk is set as (g_2, X, Y). Note that the public key comprises elements of $\mathbb{G}_2$, while the secret key comprises elements of $\mathbb{Z}_q^*$.
- **PS.Sign**$(params, sk, m) \rightarrow (\sigma)$: For a message $m \in \mathbb{Z}_q$, pick a random $h \in \mathbb{G}_1^*$. Outputs a signature $\sigma = (\sigma_1, \sigma_2) = (h, h^{x+y\cdot m})$.

- **PS.Verify**$(params, pk, m, \sigma) \rightarrow (true/false)$: It parses $\sigma = (\sigma_1, \sigma_2)$ and $pk = (g_2, X, Y)$. Output true if $\sigma_1 \neq 1_{\mathbb{G}_1}$ and $e(\sigma_1, XY^m) = e(\sigma_2, g_2)$; otherwise output false.

The PS signature enables users to obtain credentials for a private attribute. Instead of sharing attribute m with authority, the user calculates a commitment $C = g_1^t Y_1^m$, where $Y_1 = g_1^x$ and $t \in \mathbb{Z}_p$. The user then sends C and a zero-knowledge proof of opening C to the authority. Once the authority verifies the proof, it selects $u \xleftarrow{R} \mathbb{Z}_p$ and generates a blinded credential $\sigma' = (g_1^u, (X_1 C)^u)$, where $X_1 = g_1^x$, and sends it to the user. The user unblinds the credential using blind factor t as follows: $\sigma = \sigma_1 = g_1^u$ and $\sigma_2 = (X_1 C)^u / g_1^{ut} = (X g_1^t Y_1^m / g_1^t)^u = (X_1 Y_1^m)^u$. This value then serves as the credential. A detailed scheme is presented in Appendix A.

4.2 Zero-knowledge Proxy Re-identification

The Zero-knowledge Proxy Re-identification (ZK-PRI) [11] transforms (i.e., re-identifies) an identification protocol for a delegator (Alice) into one for a delegate (Bob) using a re-proof key [11]. The ZK-PRI [11] consists of two zero-knowledge identification (ZKI) protocols and an PRI.RProof algorithm. The PRI.RProof algorithm enables the proxy to convert a transcript of the first ZKI protocol into another one for two different public keys. The first ZKI protocol $\mathcal{I}_1$ (Section B.1) is a standard Schnorr's [40] protocol based on the Discrete Logarithm (DL) assumption (def: 3), while the second ZKI protocol $\mathcal{I}_2$ (Section B.2) is a bilinear variant of Schnorr's identification protocols, as detailed in the Appendix (B). The proxy has a re-proof key computed by the algorithm PRI.RGen.

- **PRI.KeyGen**: In protocol $\mathcal{I}_1$, the public key $pk_1 = (h_1, h_2) = (g_1^{sk_1}, g_2^{1/sk_1})$ is a BDLV instance (def: 5) in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$, with the private key $(sk_1 \xleftarrow{R} \mathbb{Z}_p^*)$ being the corresponding solution. In protocol $\mathcal{I}_2$, the public key $pk_2 = (g_1, g_2^{w_2})$ is an FAPI2 instance (def: 4) in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$, with the private key $(w_2 \xleftarrow{R} \mathbb{Z}_p^*)$ being the corresponding solution.
- **PRI.RGen**: The re-proof rk is computed by the delegator using the delegate public key and her private key as $rk = g_2^{w_2/sk_1}$ and share with the Proxy.
- **PRI.RProof**: The PRI.RProve is interactive algorithm between Prover, Proxy and Verifier. ① The Prover selects $a \xleftarrow{R} \mathbb{Z}_p^*$, computes $A = g_1^a$ and sends A to Proxy. ② The Proxy selects $b \xleftarrow{R} \mathbb{Z}_p^*$, computes $B = e(A, rk) \cdot e(g_1, g_2)^b$ and sends B to Verifier. ③ The Verifier select challenge $c \xleftarrow{R} \mathbb{Z}_p^*$ and sends it to Proxy which forwards it to Prover. ④ The Prover computes the response for sk_1 as $\alpha = a + sk_1 \cdot c$ and sends it to Proxy. ⑤ The Proxy checks if $g_1^\alpha = A \cdot h_1^c$, then computes $\beta = g_2^b \cdot rk^\alpha$ and sends β to Verifier.
- **PRI.Verify**: The Verifier checks if $e(g_1, \beta) = B \cdot pk_2^c$ holds.

4.2.1 Limitation. The ZK-PRI (Section 4.2) is a delegated identification protocol, not a delegated anonymous credential presentation and authentication protocol. In the ZK-PRI, the proxy transforms one identification protocol into another by re-identifying the delegate (Alice) in place of the delegator (Bob) using the re-proof key rk. The ZK-PRI construction does not support proof of knowledge of anonymous credentials, especially with selective disclosure, unlinkability, and proofing a predicate based on attributes embedded in anonymous credentials, and therefore does not achieve our design goals. The limitation comes from the PRI.RGen and PRI.RProof algorithms - since the $rk = g_2^{w_2/sk_1}$ is computed by the delegator using his secret key sk_1. In the PRI.RProve the proxy can only use the rk to prove knowledge of delegator sk_1, which was employed in constructing the rk. As a result, in ZK-PRI, the proxy cannot generate zero-knowledge proof of knowledge of any general-purpose anonymous credentials, which require knowledge of attributes embedded in the credential and are issued by an independent third-party certifying authority.

In our SilentProof, we leverage the RPI.RGen algorithm to construct a uni-directional re-proof key rk and enable the blockchain (acting as a proxy) to not only re-identify the user using the rk but extend it to delegate and generate a zero-knowledge proof of knowledge of user anonymous credentials with selective disclosure without learning about the underlying user private attributes.

5 SILENTPROOF

5.1 Overview

The SilentProof is designed to be compatible with well-known general-purpose anonymous credentials issuance such as single issuance Pointcheval and Sanders (PS) signature [37] or threshold issuance Coconut signature [41]. These credentials (signatures) are short, re-randomizable, and consist of only two group elements, regardless of the number of attributes. In the SilentProof protocol, users obtain anonymous credentials either through PS [37] or Coconut signature [41]. Users then outsource their issued credentials using the Credential Outsourcing scheme of SilentProof as a one-time setup process. This involves credential embedding attribute m, a commitment of attribute m, a re-proof key rk obtained through zero-knowledge proxy re-identification [11], and proof for attribute m signed with the user's secret key. During the credential presentation, users request the Blockchain to generate a zero-knowledge proof of knowledge of credentials each time they request a service. This is done by selectively disclosing the user credential to the service provider, using the Delegable Credential Presentation scheme of SilentProof with the user's consent. Once the Blockchain receives a one-time attribute(s) disclosed permission token, it generates proof to present and verify the user's credentials to the designated service provider. The rk generated using the user secret key and blockchain public key allows the blockchain to transform (or re-identify) the user-provided proof for attribute m along with re-randomizing the blinded PS or Coconut signatures without learning about attribute actual value m to preserve user privacy and unlinkable selective attribute revelations.

5.2 Algorithm Design of SilentProof

This section introduces the SilentProof framework, emphasizing the communication flow illustrated in Figure 2. Additionally, it provides definitions for the Credential Outsourcing and Delegable Credential Presentation schemes within SilentProof.

Credential Outsourcing (CO) Scheme. The CO scheme enables users to securely delegate their certified credentials and attributes

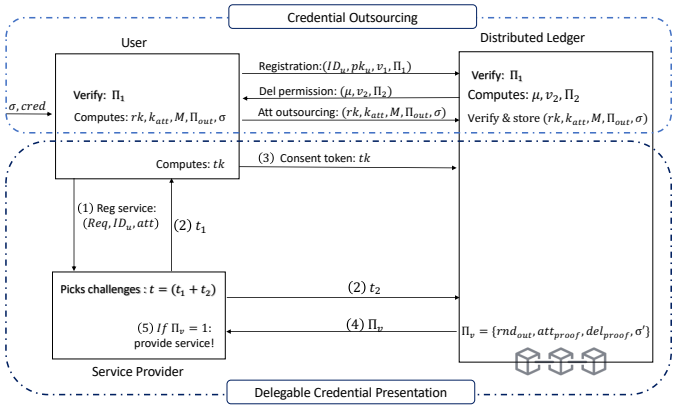

Figure 2: Framework of SilentProof

to the blockchain, facilitating efficient and trusted verification processes with various service providers. To initiate the CO scheme, a user registers on the blockchain and requests delegation permission. Upon acceptance, the blockchain validator issues the user a delegation permission token μ. Using the blockchain's public key pk_b and the user's secret key sk_u, the user generates a delegation or re-proof key rk during system initialization. This key allows the blockchain to perform re-identification on behalf of the user when interacting with verifiers. After registration, the user generates an attribute verification key k_{att} for each attribute and creates a proof of knowledge to demonstrate possession of the attribute's private value m and secret key. The delegation permission token μ is associated with the attribute verification key k_{att}. Once the blockchain verifies the proof, it stores the attribute verification key k_{att}, attribute commitment, and corresponding proof, ensuring their secure retention.

Definition 1. *The CO scheme comprises of the Setup, KeyGen, ClientReg, RKGen, AKGen, CO.Prove, CO.Verify algorithms.*

- **Setup(1^λ) $\rightarrow$ ($params$):** It defines the system parameters $params$ based on the security parameter λ, and $params$ are publicly available.
- **KeyGen($params$) $\rightarrow$ (sk, pk):** It runs by the user and delegator to generate their secret key sk and public key pk from the public $params$.
- **ClientReg($\mathcal{D}, \mathcal{L}, pk_u, pk_b$) $\rightarrow$ (μ, ID_u):** The client registration algorithm ClientReg is a two-party interactive algorithm between the user and the delegator (blockchain). The user initiates the registration process and requests a delegator $\mathcal{D}$ for a delegation permission token μ to receive delegation services and registers with blockchain ledger $\mathcal{L}$ with respective public key Pk_b. Upon accepting the delegation request, the delegator generates μ, which is tied to a secret value s determined collaboratively by the user and the delegator, and issues registered user a pseudonymous identifier ID_u.
- **RKGen(sk_u, pk_b) $\rightarrow$ (rk):** The re-proof key generation algorithm RKGen is run by the user to generate a re-proof key rk for the delegator $\mathcal{D}$ using the user's secret key sk_u and blockchain public key pk_b.
- **AKGen(τ, s_1, μ, k) $\rightarrow$ (k_{att}):** The attribute key generation algorithm AKGen is run by the user, and it takes as input a randomly generated secret value $\tau \xleftarrow{R} \mathbb{Z}_p^*$, user partial secret share s_1, the delegation permission token μ, and the hash of the attribute public value $k = H_1(att)$ and outputs the attribute verification key k_{att}.
- **CO.Prove $\rightarrow$ (π_{co}):** It runs by the user to compute a proof (π_{co}) of possession of a credential certifying the private attribute m, possession of m itself, and corresponding attribute private key β.
- **CO.Verify ($\pi_{co}, \pi_{sign}, pk_{ca}, pk_{u,1}$) $\rightarrow$ ($true/false$):** It runs by the delegator $\mathcal{D}$ to verify a credential embedding a private attribute m, a proof of knowledge of signature π_{sign}, a proof for attribute m, using credential issuer public key pk_{ca}, user public key pk_u.

Delegable Credential Presentation (DCP) Scheme. The DCP scheme enables re-identification and delegation of proof of knowledge of a signature (credential). It allows the blockchain to represent the user through two tasks: re-identification using a re-proof key rk and verification of the user's certified credential containing encoded user attribute(s) m. Additionally, the DCP requires the blockchain validator to obtain the user's consent, represented by a one-time attribute token (tk), before generating proof for any service provider on behalf of the user.

Definition 2. *The DCP scheme consists of AuthCommit, AuthConsent, DCP.Prove, DCP.Verify algorithms.*

- **AuthCommit(M, R, rk) $\rightarrow$ (S, A, B):** The authentication commitment algorithm AuthCommit is run by the delegator, and it takes as input the user pre-provided commitments M and R and the re-proof key rk and outputs new commitments, denoted as authentication commitments (S, A, B).
- **AuthConsent(ID_u, att, k, req) $\rightarrow$ (tk):** The authentication consent algorithm AuthConsent is a three-party computation process between the user, delegator, and verifier (service provider) to generate one-time attribute token tk. The user initiates the process by sending a service request req, a user identifier ID_u, and the attribute public value or identifier (att), and a hash value for att, $k = H_2(att)$ to the verifier. The verifier selects a random challenge $t = (t_0 + t_1)$ from $\mathbb{Z}_p^*$ and sends t_0 to the delegator and t_1 to the user. The user then generates the one-time attribute token tk upon receiving t_1 and sends tk to the delegator for generating proof for the credential embedding m.
- **DCP.Prove($\sigma, M, R, P, S, A, B, rk, tk, k_{att}, t_0$) $\rightarrow$ (x, y, θ, T):** It runs by the delegator and takes as input user pre-provided credential σ certifying attribute m, commitments (M, R), proof P for m signed with user secret key, delegator generated commitments (S, A, B), re-proof key rk, one-time attribute token tk, attribute verification key k_{att}, and challenge t_0, and outputs authentication proof $\pi_v = (x, y, \theta, T, z_t)$. The delegator sends $\Theta = (S, A, B, t_2, N, N', c_2, \pi_v, \sigma')$ to verifier.
- **DCP.Verify(Θ, pk_b, k) $\rightarrow$ ($true/false$):** It runs by verifier and takes as input the Θ, delegator (blockchain) public key pk_b, and attribute hash value k as input and output true if proof holds otherwise, output false.

6 A CONCRETE CONSTRUCTION

6.1 Credential Outsourcing Scheme

Setup(1^λ) → ($params$): Choose a bilinear group $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ with order p, where p is a λ-bit prime number, and e is computationally efficient bilinear map function that maps elements from $\mathbb{G}_1$ and $\mathbb{G}_2$ to $\mathbb{G}_T$. Let g_1 be generators of $\mathbb{G}_1$ and g_2 be generator of $\mathbb{G}_2$. Let $H : \{0,1\}^* \to \mathbb{Z}_p^*$ be a hash function that maps a commitment message into an element in $\mathbb{Z}_p^*$. Let $H_1 : \{0,1\}^* \to \mathbb{G}_2$ be a hash function that maps attribute public value att to an element in $\mathbb{G}_2$. Let SilentProofID be an identity for the Blockchain and $\mathcal{L} = H_1(SilentProofID)$. The system parameters are $params = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e, H, H_1, \mathcal{L})$. The (h_0, h_1) are group elements computed through a refactoring process in CO.AttProve algorithm (Section 6.1) and publicly available.

KeyGen($params$) → (sk, pk): The KeyGen algorithm in our protocol is based on the work by Bultel et al. [11], generates user and delegator key pairs using distinct cryptographic assumptions. This approach enables the construction of a secure unidirectional proxy re-identification key, denoted as rk, utilizing the user's secret key and the delegator's public key. For the user, the public key pk_u is derived from the BDLV assumption (Def: 5) in the bilinear group $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$, with the corresponding secret key sk_u as the solution. Conversely, the public key pk_b of the blockchain is derived from the FAPI2 assumption (Def: 4) in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$, where the corresponding secret key sk_b serves as the solution. To generate the user's secret key sk_u and public key pk_u, the user selects $\alpha \xleftarrow{R} \mathbb{Z}_p^*$, while the blockchain selects $w \xleftarrow{R} \mathbb{Z}_p^*$, resulting in the following key pairs:

$$sk_u = \alpha, \quad pk_u = (pk_{u,1}, pk_{u,2}) = (g_1^{\alpha}, g_2^{1/\alpha})$$
$$sk_b = w, \quad pk_b = (pk_{b,1}, pk_{b,2}) = e(g_1, g_2^{w})$$

ClientReg($\mathcal{L}, pk_u, pk_b$) → (μ, ID_u): Users register with the blockchain $\mathcal{L}$ to request the blockchain's assistance acting as their representative and performing delegated tasks on their behalf. Once the blockchain approves the user's delegation request, it issues a delegation permission token μ and assigns a pseudonymous identifier ID_u to authorize its representative status. Subsequently, the user can securely outsource their credentials, encrypted attributes, and proof of possession of certified attributes to the blockchain.

A user initiates client registration by randomly choosing a secret $s_1 \xleftarrow{R} \mathbb{Z}_p^*$, computes commitment $v_1 = g_1^{s_1}$, and generates a NIZKP proof π_{s_1} as:

$$\pi_{s_1} = PK\{s_1 : v_1 = g_1^{s_1}\}$$

user sends proof π_{s_1} to blockchain. Blockchain verifies π_{s_1} and if holds, computes delegation permission token μ by randomly choosing $(s_2, o) \xleftarrow{R} \mathbb{Z}_p^*$: Let $s_0 = s_1 + s_2$, it computes $v_2 = g_1^{s_2}$, $v_1 \cdot g_1^{s_2} = g_1^{s_1+s_2} = v$, and $\mu = (g_1^{s_0} h)^{1/(\mathrm{sk}_b \cdot o)}$. It then computes a NIZKP proof for π_{s_2} as:

$$\pi_{s_2} = PK\{(s_2, o, sk_b) : v_2 = g_1^{s_2} \wedge \mu^{sk_b} = v \cdot h \cdot \mu^{-o}\}$$

The delegator generates a pseudonymous identifier ID_u for the user associated with μ and sends $(\mu, ID_u, \pi_{s_2}, v_2)$ to the user and store (μ, s_2, v) associated to ID_u on ledger. The user verifies π_{s_2}, calculates $v = g_1^{s_1} \cdot g_1^{s_2}$, and utilizes μ in the AKGen algorithm to compute the attribute verification key. The μ is computed based on the Boneh-Boyen signature [7] (without pairing version) and is proven existentially unforgeable under weak chosen message attack under the q-Strong Diffie-Hellman assumption in the random oracle model [2].

RKGen(sk_u, pk_b) → (rk): The user initiates the re-proof key generation algorithm RKGen to generate the re-proof key rk by incorporating the blockchain public key pk_b and the user's secret key sk_u, computes the re-proof key as $rk = (g_2^{w})^{1/\alpha} = g_2^{w/\alpha}$. The generated re-proof key in this process is unidirectional, meaning it cannot be transferred to another entity as it is derived solely from the blockchain's public key and the user's secret key. The construction of the re-proof key (rk) is based on the zero-knowledge proxy re-identification protocol presented in [11], which has proven that collusion between the delegator and verifier does not expose the delegate's secret key.

AKGen(τ, s_1, μ, k) → (k_{att}): The user initiates the attribute key generation algorithm AKGen to generate an attribute verification key k_{att} and private key β for each certified attribute m. Each attribute in SilentProof consists of a public value, denoted as (att), and a corresponding private value, denoted as (m). The (m) is a randomly selected group element in Z_p^*. The public value (att) represents a general category or label (e.g., "Date of Birth"), while the private value (m) represents the actual date of birth of the individual (e.g., "January 1, 1990"). It computes as follows:

$$k = H_1(att), \quad \beta = (\tau + \mu \cdot s_1), \quad k_{att} = k^{\beta/\alpha}, \quad \tau \xleftarrow{R} \mathbb{Z}_p^*$$

The attribute verification key k_{att} serves two essential roles during a credential presentation. First, it enables the blockchain to verify the requesting user's identity as a registered user, ensuring that the attributes are securely stored and associated with the delegation permission token (μ). Second, along with the one-time attribute token tk, it ensures that the blockchain obtains the user's consent before generating proof of knowledge and disclosing specific attributes to service providers. Using k_{att}, the protocol safeguards against potential disruptions from malicious users or the blockchain itself, thereby preserving protocol integrity.

CO.Prove → (π_{co}): The CO.Prove consists of the following PS.Prove and CO.AttProve sub-algorithms:

- **PS.Prove (σ, m) → (σ', π_{sign}):** The user utilizes PS.Prove algorithm, to generate proof of knowledge of PS signature $\sigma = (\sigma_1, \sigma_2) = (h, h^{\gamma_1 + \gamma_2 \cdot m})$, which certifies the private value (m) associated with the attribute. In the PS.Prove algorithm, the user first re-randomizes and blinds σ by selecting random values $(r_0, t_0) \xleftarrow{R} \mathbb{Z}p^*$. This results in a modified signature $\sigma = (\sigma_1', \sigma_2') = (\sigma_1^{r_0}, (\sigma_2 \cdot \sigma_1^{t_0})^{r_0})$. Finally, the user generates a Schnorr-based zero-knowledge proof of knowledge (π_{sign}) for m and t to establish the validity of the proof as:

$$\pi_{sign} = PK(m, t_0) : e(\sigma_1', X) \cdot e(\sigma_1', Y)^m \cdot e(\sigma_1', g_2)^t = e(\sigma_2', g_2) \tag{1}$$

  where $(X, Y) = (g_2^{\gamma_1}, g_2^{\gamma_2})$ and issuer public key is $pk_{ca} = (g_2, X, Y)$ and $sk_{ca} = (\gamma_1, \gamma_2) \xleftarrow{R} \mathbb{Z}_p^*$ issuer private key, r_0 randomization and t_0 blinding factor. To preserve the multi-show unlinkability of attribute revelation, the PS signature must be re-randomized every time it is presented to the

verifier. However, in the CO scheme, where the user only outsources the PS signature once, there is no need to perform the re-randomization step.

- **CO.AttProve** $(m, sk_u, \beta, k_{att}, k,) \rightarrow (M, R, P, \pi_{co})$**:** Users outsource their attributes to the blockchain by computing a commitment (M) for each attribute's private value (m). They then generate a proof of possession for m and produce a proof (P). The blockchain later utilizes P to generate a delegation proof for m. Notably, as m is not directly shared with the blockchain, only its commitment M and proof P and token tk are used, enabling the blockchain to establish proof of knowledge of m. The attribute outsourcing proof is computed as follows:
  To ensure compatibility with the PS signature proof mentioned in equation (1), a group element h_1 for the attribute commitment is computed using equation (1). We can refactor equation (1) as follows:
  $$com = \frac{e(\sigma_2', g_2)}{e(\sigma_1', X)} \in \mathbb{G}_T, \quad h_0 = e(\sigma_1', g_2), \quad h_1 = e(\sigma_1', Y)$$
  It is important to note that π_{sign} in equation (1) is a standard proof of knowledge of an opening of the commitment com for generators h_0 and h_1, as represented in [28, 37]. That is, π_{sign} proves knowledge of an opening (m, t) such that $com = h_0^t h_1^m$, which is a Pedersen commitment. The (h_0, h_1) are added as part of $params$. While computing a commitment for attribute m, we employ the same generator h_1, resulting in $M = h_1^m$. User proof π_{co} statement below to proof knowledge of attribute m, randomization factor r, secret key $sk_u = \alpha$, attribute private key β as:
  $$\pi_{co} = PK\{(m, r, \alpha, \beta) : M = h_1^m \wedge R = g_1^r \\ \wedge\ pk_{u,1} = g_1^\alpha \wedge k_{att} = k^{\beta/\alpha}\}$$
  As the π_{co} can be computed using a Schnorr-based NIZP, we outline the proof to provide an understanding of how we derive the value P that the user submits to the blockchain during the outsourcing process. This value is subsequently utilized by the blockchain when generating the delegation proof.
  To prove π_{co}, user randomly chooses $(m', \alpha', \beta', r') \xleftarrow{R} \mathbb{Z}_p^*$, computes commitments, challenges and prove responses as:
  computes commitments:
  $$M' = h_1^{m'}, \quad pk_{u,1}' = g_1^{\alpha'}, \quad k_{att}' = k^{\beta'/\alpha'}, \quad R' = g_1^{r'}$$
  computes challenges:
  $$c = H(M, M'), \quad f = H(R, R')$$
  $$d = H(pk_{u,1}, pk_{u,1}'), \quad e = H(k_{att}, k_{att}')$$
  computes responses:
  $$z_m = m' + m \cdot c, \quad z_r = r' + r \cdot f, \quad z_\alpha = \alpha' + \alpha \cdot d,$$
  $$z_\beta = \beta'/\alpha' + \beta/\alpha \cdot e, \quad P = m + r + \alpha \cdot d$$

User sends $\Pi_{co} = (\sigma', P, (M, M', R, R', pk_{u,1}', k_{att}, k_{att}'), (c, d, e, f), (z_m, z_r, z_\alpha, z_\beta))$ to blockchain.

CO.Verify $(\pi_{co}, \pi_{sign}, pk_{ca}, pk_u) \rightarrow (true/false)$**:** The blockchain verifies the PS signature using PS.Verify algorithm and checks if proofs π_{co}, π_{sign} hold. If successful, it stores the commitment M and the proof P for m, along with the user's information (ID_u, μ, pk_u), in the ledger.

- **PS.Verify** $(\sigma', \pi_{sign}, pk_{ca}) \rightarrow (true/false)$**:** Using PS.Verify delegator parse $\sigma' = (\sigma_1', \sigma_2')$, verify π_{sign} using pk_{ca} and σ' and outputs $true$ if $\sigma_1' \neq 1_{G_1}$ and $e(\sigma_1', X \cdot Y^m) = (\sigma_2', g_2)$.
- **CO.AttVerify** $(\pi_{co}) \rightarrow (true/false)$**:** Re-calculate challenges $c = H(M, M')$, $d = H(pk_{u,1}, pk_{u,1}')$, $e = H(k_{att}, k_{att}')$, $f = H(R, R')$, and check if following statements hold:
  $$h_1^{z_m} = M' \cdot M^c, \quad g_1^{z_r} = R' \cdot R^f$$
  $$g_1^{z_\alpha} = pk_{u,1}' \cdot pk_{u,1}^d, \quad k^{z_\beta} = k_{att}' \cdot k_{att}^e$$

6.2 Delegable Credential Presentation Scheme

AuthCommit$(M, R, rk) \rightarrow (S, A, B)$**:** Blockchain chooses $(s, a, b) \in Z_p^*$ and computes authentication commitments (S, A, B) as:

$$S = e(M \cdot R, rk) \cdot e(h_1 \cdot g_1, g_2)^s$$

$$A = e(M \cdot R, g_2^a), \quad B = e(h_1 \cdot g_1, g_2)^b$$

AuthConsent$(ID_u, att, k, req) \rightarrow (tk)$**:** The user sends the information $(service, ID_u, att)$ to the service provider in the form of a QR code to request the desired service. The service provider (verifier) randomly chooses $t = (t_0 + t_1) \xleftarrow{R} \mathbb{Z}_p^*$, and sends t_1 to user and t_0 along with (ID_u, att) to blockchain. The user generates a one-time attribute token as $tk = k^{(t_1 - \beta)/\alpha}$ and sends tk to the blockchain to authorize authentication proof.

DCP.Prove$(\sigma, M, R, P, S, A, B, rk, tk, k_{att}, t_0) \rightarrow (x, y, \theta, T)$**:** In this algorithm, blockchain generates a zero-knowledge proof of knowledge of user credentials to the verifier. Using the re-proof key rk, attribute key k_{att}, and user-provided consent token tk, blockchain can *transform* the proofs (P, M, R) given during the CO phase into another proxy-based zero-knowledge proof (x, y, θ, T) that prove possession of user's attributes in their credentials. First, to render the scheme non-interactive, blockchain generates a new challenge t_2 as $t_2 = H(t_0 || H(S, A, B))$. This combination technique is similar to the one described in [16]. Then, computes response (x, y, θ, T):

$$x = g_2^a \cdot rk^{t_2}, \quad y = g_2^{b + s \cdot t_2}$$

$$\theta = g_2^s \cdot (rk \cdot tk \cdot k_{att})^P, \quad T = e(M \cdot R, tk \cdot k_{att})$$

Finally, randomize and blind PS signature as $\sigma = (\sigma_1', \sigma_2') = (\sigma_1^{r'}, (\sigma_2 \cdot \sigma_1^t)^{r'})$ where $(r', t) \in Z_p^*$ and r' is randomize factor and t is blinding factor. In proving knowledge of PS signature, it is also required to prove blinding factor t and attribute m. Hence, a Schonrr-based NIZK for t can be generated by choosing $t' \in Z_p^*$:

$$N = {h_0}^t, \quad N' = h_0^{t'}, \quad c_2 = H(N, N'), \quad z_t = t' + t \cdot c_2$$

blockchain sends $\Theta = ((S, A, B, N, N'), (x, y, \theta, T, z_t), (t_2, c_2))$ to service provider.

In the interactive version of DCP.Prove; the blockchain follows the standard Sigma zero-knowledge proof of knowledge process. This involves computing the challenge (S, A, B) and sending it to the verifier. The blockchain then receives a challenge t_0 from the verifier and computes the corresponding zero-knowledge proof of knowledge response (x, y, θ, T), which is returned to the verifier.

DCP.Verify$(\Theta, \mathrm{pk}_b, k) \rightarrow (true/false)$**:** Verifier checks if the following equations hold by first re-calculating the challenges

$t_2 = H(t_0||H(S, A, B))$, and $c_2 = H(N, N'$, and checks:

$$e\left(h_1 \cdot g_1, \theta\right) = S \cdot e\left(h_1 \cdot g_1, pk_b\right)^d \cdot e\left(g_1, k\right)^{(d \cdot t_1)} \cdot T \quad (2)$$

$$e\left(M \cdot R, x\right) \cdot e\left(h_1 \cdot g_1, y\right) = A \cdot B \cdot S^{t_2} \quad (3)$$

$$h_0^{z_t} = N' \cdot N^{c_2} \quad (4)$$

The expression $M \cdot N = h_1^m \cdot h_0^t$ still yields a Pedersen commitment that remains both hiding and binding. Finally, verifies PS signature by parsing $\sigma' = (\sigma_1', \sigma_2')$, and $pk_{ca} = (X, Y) = (g_2^{\gamma_1}, g_2^{\gamma_2}, g_2)$ and checks if:

$$\sigma_1' \neq 1_{\mathbb{G}_1}, \quad e(\sigma_1', \gamma_1 \cdot M \cdot N) = e(\sigma_2' \cdot N, g_2) \quad (5)$$

The Fiat-Shamir transformation employs a hash function H to generate an unpredictable challenge post-commitment. The weak Fiat-Shamir (wFS) only hashes the commitment and is vulnerable to malicious provers choosing their problem instance adaptively [6]. However, the strong Fiat-Shamir (sFS) hashes both the commitment and statement, ensuring adaptive security [6]. In our proposed DCP.Prove algorithm, since the commitments $S = e(M \cdot R, rk) \cdot e(g_1, g_2)^s$ and $A = e(M \cdot R, g_2^a)$ are initially computed by embedding the proving statement M and R. The challenge t_2 is then computed by hashing (S, A, B), where S and A include the proving statement M and R. This infers that the challenge includes both the commitments and the proving statement, and the S and R cannot be computed without the proving statements (M, R), making the non-interactive variant of our scheme adaptively secure against malicious provers.

6.3 Multi-Attributes SilentProof

We extend our scheme to support the delegation of anonymous authentication for credentials with multiple attributes. However, it is essential to note that SilentProof's primary focus is on the presentation of anonymous credentials, not on their issuance. Therefore, to present certified credentials with multiple attributes, the issuance protocol responsible for credentials must support the issuance of credentials with multiple attributes. The issuance protocols, both based on the PS signature [37] and Coconut [41], support issuing credentials with multiple attributes where credential size remains two group elements regardless of the number of embedded attributes in a credential or number of issuance authorities (in case of Coconut scheme). The credential generalizes as follows:

$$\sigma = (\sigma_1, \sigma_2) = (h, h^{x + \Sigma_{i=1}^q m_i y_i})$$

We present a detailed Multi-Attribute SilentProof scheme in Appendix (E) that supports delegable anonymous credential presentation with multiple attributes.

7 SECURITY ANALYSIS OF SILENTPROOF

This section describes the security proof based on the complexity assumption and security definitions in Appendix D.1.

7.1 Security Proof of SilentProof Protocol

Theorem 1. *Under LRSW [33] and XDH [8] assumptions, SilentProof ensures security, unforgeability, blindness, and unlinkability. It also exhibits soundness, completeness, and honest-verifier zero-knowledge based on DL, BDLV, and FAPI2 assumptions.*

Proof. SilentProof's security relies on underlying components like the PS signature [37], CO, and DCP schemes. It ensures unforgeability, blindness, unlinkability, and zero-knowledge, even against (verifier and blockchain) or (verifier and credential issuer) collusion. A proof outline is in Appendix D.4. □

7.2 Security Proofs of CO Scheme

Theorem 2. *The CO scheme is a complete, sound, and (honest verifier) zero-knowledge proof of knowledge under the DL assumption.*

Proof. The CO scheme comprises the I_1 = Setup, KeyGen, Client, RKGen, AKGen, CO.Prove, CO.Verify algorithms. Note that our ClientReg algorithm is based on the without pairings variant of Boney-Boyen that is proven zero-knowledge proof of knowledge and unforgeability of underlying signature by Arfaoui et al., [2].

Completeness. We deduce that I_1 is complete in Theorem 2. Suppose the prover (user) knows its secret key $sk_u = \alpha$, attribute key β and attribute private value m for attribute att and the verifier (blockchain) knows prover public key pk_u, attribute verification key k_{att} and a commitment M for attribute m and verifier honestly run the protocol I_1. First, prover sends the transcript $\Pi_1 = ((k_{att}, M, M', R, R', pk_{u,1}', z_m, z_r z_\alpha, z_\beta)$ to verifier. Then verifier computes $c = H(M, M')$, $c_1 = H(R, R')$, $d = H(pk_{u,1}, pk_{u,1}')$, $e = H(k_{att}, k_{att}')$ and verify following equations if holds then outputs b = 1.

$$h_1^{z_m} = h_1^{m' + m \cdot c} = g_1^{m'} \cdot g_1^{m \cdot c} = M' \cdot M^c$$

$$g_1^{z_r} = g_1^{r' + r \cdot f} = g_1^{r'} \cdot g_1^{r \cdot f} = R' \cdot R^f$$

$$g_1^{z_\alpha} = g_1^{\alpha' + \alpha \cdot d} = g_1^{\alpha'} \cdot g_1^{\alpha \cdot d} = pk_{u,1}' \cdot pk_{u,1}^d$$

$$k^{z_\beta} = k^{\beta'/\alpha' + \beta/\alpha \cdot e} = k^{\beta'/\alpha'} \cdot k^{\beta/\alpha \cdot e} = k_{att}' \cdot k_{att}^e$$

We deduce CO.Verify returns 1, which concludes completeness proof.

Soundness. We show the interactive version of I_1 is valid and sound. Let λ be an integer and CO.Prove (P^*) and CO.Verify (V) are PPT algorithms, and bit is a bit-string, and ϵ be a function as:

$$\Pr\left[b \leftarrow out_V(I_1\langle P^*(bit); V(M, R, pk_u, k_{att}\rangle) : b = 1\right] = \epsilon(\lambda)$$

To establish knowledge soundness for I_1, we construct a PPT knowledge extractor $\mathcal{K}$ that treats CO.Prove and CO.Verify as proof of knowledge. $\mathcal{K}$ interacts with the arbitrary prover (potentially dishonest) using the oracle CO.Prove* as follows:

(1) $\mathcal{K}$ receives the commitment $(M', R', pk_{u,1}', k_{att}')$.

(2) $\mathcal{K}$ chooses $(c_0, d_0, e_0, f_0) \xleftarrow{R} \mathbb{Z}_p^*$, and sends it to the oracle and receives $(z_{m,0}, z_{r,0}, z_{\alpha,0}, z_{\beta,0})$. $\mathcal{K}$ rewinds the oracle to its state before step 2. Then $\mathcal{K}$ chooses $(c_1, d_1, e_1, f_1) \xleftarrow{R} \mathbb{Z}_p^*$, and sends it to the oracle and receives $(z_{m,1}, z_{r,1}, z_{\alpha,1}, z_{\beta,1})$.

Finally, $\mathcal{K}$ extracts witness $(m, r, \alpha, \beta/\alpha)$ from the two accepting conversations $(z_{m,0}, z_{\alpha,0}, z_{\beta,0})$, $(z_{m,1}, z_{\alpha,1}, z_{\beta,1})$ as follows:

$$m = \frac{z_{m,1} - z_{m,0}}{c_1 - c_0}, \quad \alpha = \frac{z_{\alpha,1} - z_{\alpha,0}}{d_1 - d_0},$$

$$\beta/\alpha = \frac{z_{\beta,1} - z_{\beta,0}}{e_1 - e_0}, \quad r = \frac{z_{r,1} - z_{r,0}}{f_1 - f_0}$$

We show that m is the private attribute value to M, r is the private value of randomness to R, α is the secret key corresponding to

the user's public key pk_u, and β/α is the attribute private key corresponding to attribute verification key k_{att}. If $h_1^{z_m} = M' \cdot M^c$, $g_1^{z_\alpha} = pk'_{u,1} \cdot pk_{u,1}^d$, $k^{z_\beta} = k'_{att} \cdot k_{att}^e$, $g_1^{z_r} = R' \cdot R^f$ and $c_0 \neq c_1$, $d_0 \neq d_1$, $e_0 \neq e_1$, $f_0 \neq f_1$ then:

$$h_1^m = h_1^{\frac{(z_{m,1}-z_{m,0})}{(c_1-c_0)}} = \left(\frac{h_1^{z_{m,1}}}{h_1^{z_{m,0}}}\right)^{\frac{1}{c_1-c_0}} = \left(\frac{M' \cdot M^{c_1}}{M' \cdot M^{c_0}}\right)^{\frac{1}{c_1-c_0}} = M$$

$$g_1^r = g_1^{\frac{(z_{r,1}-z_{r,0})}{(f_1-f_0)}} = \left(\frac{g_1^{z_{r,1}}}{g_1^{z_{r,0}}}\right)^{\frac{1}{f_1-f_0}} = \left(\frac{R' \cdot R^{c_1}}{R' \cdot R^{c_0}}\right)^{\frac{1}{f_1-f_0}} = R$$

$$g_1^\alpha = g_1^{\frac{(z_{\alpha,1}-z_{\alpha,0})}{(d_1-d_0)}} = \left(\frac{g_1^{z_{\alpha,1}}}{g_1^{z_{\alpha,0}}}\right)^{\frac{1}{(d_1-d_0)}} = \left(\frac{pk'_{u,1} \cdot pk_{u,1}^{d_1}}{pk'_{u,1} \cdot pk_{u,1}^{d_0}}\right)^{\frac{1}{(d_1-d_0)}} = pk_{u,1}$$

$$k^{\beta/\alpha} = k^{\frac{(z_{\beta,1}-z_{\beta,0})}{(e_1-e_0)}} = \left(\frac{k^{z_{\beta,1}}}{g_1^{z_{\beta,0}}}\right)^{\frac{1}{(e_1-e_0)}} = \left(\frac{k'_{att} \cdot k_{att}^{d_1}}{k'_{att} \cdot k_{att}^{d_0}}\right)^{\frac{1}{(e_1-e_0)}} = k_{att}$$

The detailed probability distribution for knowledge soundness is present in Appendix (D.2). The non-interactive case of I_1 can be proven to have knowledge soundness, similar to the interactive case. In this scenario, challenges $(c_0, c_1, d_0, d_1, e_0, e_1, f_0, f_1)$ are generated from a random oracle using the corresponding commitments $(M, M', R, R', pk_{u,1}, pk'_{u,1}, k_{att}, k'_{att})$ as inputs. This prevents the adversary from knowing the challenges beforehand, as the commitments are part of the random oracle's input. Furthermore, the random oracle selects the challenges in a manner consistent with the verifier in the interactive proof system. Thus, both the interactive and non-interactive versions of I_1 are considered sound, completing the soundness proof.

Honest-verifier Zero-knowledge. We prove that I_1 is honest verifier zero-knowledge (HVZK) by building a PPT algorithm Simulator (Sim) such that Sim(M, R, pk_u, k_{att}) and CO.Prove(m, M, r, R, α, pk_u, β/α, k_{att}) follow the same probability distribution. Intuitively, we can efficiently simulate conversations between CO.Prove and CO.Verify without knowing the witnesses $(m, \alpha, \beta/\alpha)$.

- The *Sim* randomly chooses $(z_m, z_r, z_\alpha, z_\beta) \xleftarrow{R} \mathbb{Z}_p^*$ and (c, d, e, f) $\xleftarrow{R} \mathbb{Z}_p^*$, and computes:

$$M' = h_1^{z_m}/M^c, \quad R' = g_1^{z_r}/R^f$$
$$pk'_{u,1} = g_1^{z_\alpha}/pk_{u,1}^d, \quad k'_{att} = k^{z_\beta}/k_{att}^e$$

- Sim outputs $\Pi = (M', R', k'_{u,1}, k'_{att}, z_m, z_\alpha, z_\beta, c, d, e)$ which is a valid proof since

$$h_1^{z_m} = M' \cdot M^c, \quad g_1^{z_r} = R' \cdot R^f$$
$$g_1^{z_\alpha} = pk'_{u,1} \cdot pk_{u,1}^d, \quad k^{z_\beta} = k'_{att} \cdot k_{att}^e$$

Since $(z_m, z_r, z_\alpha, z_\beta)$ come from the uniform distribution on $\mathbb{Z}_p^*$, then the output of Sim follows the same distribution as the real protocol. Hence, it concludes that I_1 is HVZK.

□

7.3 Security Proofs of DCP Scheme

Theorem 3. *DCP scheme is a complete, sound, and honest verifier of zero knowledge under the DL, BDLV, and FAPI2 assumptions.*

Proof. The DCP scheme comprises the algorithms of the I_2 = (AuthCommit, AuthConsent, DCP.Prove, DCP.Verify).

Completeness. We deduce that I_2 is complete (Theorem 3). Suppose the user knows the attribute private key β and private key $sk_u = \alpha$. The blockchain knows private key w, re-proof key $rk = g_2^{w/\alpha}$, the original commitment to user private attribute value m, a user signed attribute proof (P, R), a PS signature σ' on user attribute m with randomizing and blinding factors (r_1, t) for the PS signature, and verifier honestly run the protocol I_2. First, the user sends the (rk, M, P, R, σ) based on I_1 to the blockchain, and the blockchain verifies and randomizes the received proof, challenges, and commitments each time the blockchain generates proof on the user's behalf to a service provider (verifier) to preserve unlinkability. Then, the blockchain generates AuthCommit = (S, A, B). When each time the user wants to interact with the service provider, the AuthConsent initiated by the user and service provider picks a challenge $t = t_0 + t_1$ and sends t_1 to the user and t_0 to the blockchain. The user generates a one-time token tk by running the AuthConsent algorithm and sends tk to the blockchain. Once the blockchain receives tk, it generates proof of knowledge of user credentials that embed user attributes and sends DCP.Prove(x, y, θ, T) to the service provider. Finally, the service provider verifies the delegated proof by running DCP.Verify and checks if equations (2, 3, 4, 5) holds and outputs b = 1. Based on the correctness proof (C) show that equations (2, 3, 4, 5) hold. Hence, we conclude that the DCP.Verify algorithm returns 1, which concludes the completeness proof.

Soundness. We prove that I_2 is valid and sound. Let λ be an integer, and P* = (AuthCommit, AuthConsent, DCP.Prove) is a proving oracle and V=DCP.Verify is a verifying oracle. Let Π_v be the DCP.Prove output. Both P^* and V are PPT algorithms, and *bit* is a bit-string, and ϵ be a function such that:

$$\Pr\left[b \leftarrow out_V(I_1\langle P^*(bit); V(pk_u, pk_b, \Pi_v)\rangle) : b = 1\right] = \epsilon(\lambda)$$

We build a knowledge extractor $\mathcal{K}$ to prove knowledge soundness for the I_2 scheme. We focus on (AuthCommit, AuthConsent, DCP.Prove) and the knowledge soundness of witnesses: re-proof key rk, attribute private key β, attribute token tk, and user-signed proof P. In the interactive version, denoted as I_2, we have $P^* = (P_1^*, P_2^*)$. P_1^* represents the user as Oracle 1, P_2^* represents the blockchain as Oracle 2, and the knowledge extractor $\mathcal{K}$ acts as the verifier in the I_2 protocol. Unlike the non-interactive version of the DCP scheme (Section 6.2), the blockchain directly uses the received challenge t_0, denoted as t_2 in this proof, without creating another challenge based on the received challenge t_0 and the hash of commitments (S, A, B). The $\mathcal{K}$ is a PPT algorithm that runs the oracle P^*, which is an arbitrary prover(s) (potentially a "dishonest" one) and interacts with it as follows:

(1) $\mathcal{K}$ receives commitments (S, A, B).
(2) $\mathcal{K}$ picks $t_0 = t_{1,0} + t_{2,0}$, and sends $t_{1,0}$ to oracle 1 and $t_{1,}$ to oracle 2.
  (a) Oracle 2 receives tk_0 from oracle 1.
  (b) Oracle 2 computes $(x_0, y_0, \theta_0, T_0)$ and sends them to $\mathcal{K}$.
(3) $\mathcal{K}$ rewinds the Oracle 1 and Oracle 2 to their state before step 2. Then $\mathcal{K}$ picks $t_1 = (t_{1,1} + t_{2,1})$ and sends $t_{1,1}$ to oracle 1 and $t_{1,2}$ to oracle 2.
  (a) Oracle 2 receives tk_1 from oracle 1.
  (b) Oracle 2 computes $(x_1, y_1, \theta_1, T_1)$ and sends them to $\mathcal{K}$.

$\mathcal{K}$ drives rk as follows:

$$x_{rk} = \left(\frac{x_1}{x_0}\right)^{1/(t_{2,1}-t_{2,0})} = \left(\frac{g_2^a \cdot rk^{t_{2,1}}}{g_2^a \cdot rk^{t_{2,0}}}\right)^{1/(t_{2,1}-t_{2,0})} = rk$$

$\mathcal{K}$ computes $(k^{1/\alpha}, \beta)$ as follows: We know that $tk \cdot k_{att} = k^{(t_1-\beta/\alpha)}$. $k^{\beta/\alpha} = k^{(t_1-\beta+\beta)/\alpha} = k^{t_1/\alpha}$, we can substitute $(tk \cdot k_{att})$ with $(k^{t_1/\alpha})$ to simplify the equation below:

$$\theta_{1/\alpha} = \left(\frac{\theta_1}{\theta_0}\right)^{1/(t_{1,1}-t_{1,0})} = \left(\frac{g_2^s \cdot (rk \cdot k^{t_{1,1}/\alpha})^P}{g_2^s \cdot (rk \cdot k^{t_{1,0}/\alpha})^P}\right)^{1/(t_{1,1}-t_{1,0})}$$
$$= \left(k^{(t_{1,1}-t_{1,0})/\alpha}\right)^{1/(t_{1,1}-t_{1,0})} = k^{1/\alpha}$$

Since the attribute private key β is chosen by the user in I_1, and the purpose of multiplying tk and k_{att} in I_2 is to cancel out the attribute private key β (as the blockchain does not possess β), while still ensuring that the blockchain obtains user consent in I_2, we can derive β using the soundness rewinding procedure of I_1. By running this procedure, we obtain $(z_{\beta,1} - z_{\beta,0})/(e_1 - e_0)$, allowing us to derive β from both I_1 and I_2 as follows:

$$\theta_{1/\alpha} = k \cdot \frac{z_{\beta,1} - z_{\beta,0}}{e_1 - e_0} \quad \Rightarrow k^{1/\alpha} = k \cdot \frac{\beta}{\alpha} \Rightarrow (k^{1/\alpha})^\alpha = (k \cdot \frac{\beta}{\alpha})^\alpha = \beta$$

$\mathcal{K}$ drives $P = (m + r + \alpha \cdot d)$ as follows:

$$y_s = \left(\frac{y_1}{y_0}\right)^{1/(t_{2,1}-t_{2,0})} = \left(\frac{g_2^{b+s\cdot t_{2,1}}}{g_2^{b+s\cdot t_{2,0}}}\right)^{1/(t_{2,1}-t_{2,0})} = g_2^s$$

Substituting y_s value below derives us:

$$\begin{aligned} P &= \frac{S \cdot e(pk_u^d, g_2)}{e(h_1 \cdot g_1, y_s) \cdot e(g_1, x_{rk}) \cdot e(g_1, g_2)} \\ &= \frac{e(M.R, rk) \cdot e(h_1 \cdot g_1, g_2)^s \cdot e(g_1^{\alpha\cdot d}, g_2)}{e(h_1 \cdot g_1, g_2^s) \cdot e(g_1, rk) \cdot e(g_1, g_2)} \\ &= \frac{e(g_1, g_2)^{m+r} \cdot e(g_1, g_2)^{\alpha\cdot d}}{e(g_1, g_2)} \\ &= \frac{e(g_1, g_2)^{m+r+\alpha\cdot d}}{e(g_1, g_2)} = (m + r + \alpha \cdot d) \end{aligned}$$

We can calculate the probability distribution for each commitment similarly to the probability distribution calculation in I_1. The detailed probability distribution is presented in Appendix (D.3). We deduce that I_2 is knowledge sound.

Honest-verifier Zero-knowledge. We prove that I_2 is honest verifier zero-knowledge (HVZK) by building a PPT algorithm *Simulator (Sim)* such that *Sim* and *P* follows the same probability distribution. The *Sim* picks $L_1 = (x, y, \theta, T, z_t) \xleftarrow{R} \mathbb{Z}_p^*$, $(S, A, B, N) \xleftarrow{R} \mathbb{G}_T$, $L_2 = (c_2, t = t_1 + t_2) \xleftarrow{R} \mathbb{Z}_p^*$. The *Sim* computes as follows:

$$S = \frac{e(h_1 \cdot g_1, \theta)}{e(h_1 \cdot g_1, pk_d)^d \cdot e(g_1, k^{d\cdot t_1})}, \quad N' = h_0^{z_t}/N^{c_2}$$
$$A = \frac{e(M \cdot R, x) \cdot e(h_1 \cdot g_1, y)}{B \cdot S^{t_2}}, \quad B = \frac{e(M \cdot R, x) \cdot e(h_1 \cdot g_1, y)}{A \cdot S^{t_2}}$$

It outputs (S, B, N', L_1, L_2) which are valid transcripts since:

$$e(h_1 \cdot g_1, \theta) = S \cdot e(h_1 \cdot g_1, pk_b)^d \cdot e\big(g_1, k\big)^{(d\cdot t_1)} \cdot T$$
$$e(M \cdot R, x) \cdot e(h_1 \cdot g_1, y) = A \cdot B \cdot S^{t_2}$$
$$h_0^{z_t} = N' \cdot N^{c_2}$$

and $L_1 = (x, y, \theta, T, z_t)$ come from the uniform distribution on $\mathbb{Z}_p^*$, then the output of *Sim* follows the same distribution as the real protocol I_2. Hence it concludes the I_2 is HVZK.

□

8 PERFORMANCE EVALUATION

We developed a SilentProof prototype using Python's petlib and bplib libraries to implement primitives for the CO and DCP schemes. We selected BN-254 [5] and BLS12-381 [9] curves with OpenSSL for bilinear pairing, considering the trade-off between BN-254's performance excellence and BLS12-381's enhanced security at the 128-bit level. To optimize pairing performance, we ensure a low Hamming weight parameterization for the BLS curve, with an embedding degree of 12 [9]. See Tables (3b, 3c) for a comparison. Performance evaluation was conducted on a Google Cloud Platform e2-standard-4 instance with Intel Broadwell, 4 vCPUs, and 16 GB memory. Measurements were based on 1000 runs of each primitive, with entities running on the same instance in separate threads, excluding network latency. We assumed that client-perceived latency is bound to network latency. The proximity of blockchain nodes and service providers to the client might affect latency, with greater distances resulting in higher response times. We used PS signature [37] as an underlying signature for the credential.

8.1 Communication Overhead

Tables 2a and 2b outline the communication overhead associated with the CO scheme and DCP schemes, respectively. In these tables, the variables have the following definitions: g represents the size of a group element defined during the Setup phase, $|PK|$ corresponds to the size of the proof of knowledge for a single private value, q denotes the total number of attributes, $|S|$ indicates the size of the credential, $|ID|$ represents the size of the user identifier, and $\mathcal{O}(1)$ signifies the communication complexity involved in verifying the proof of a single attribute. Both tables provide a comprehensive overview of the communication overhead between each participating entity, considering the relevant primitives within each scheme. Table 3a shows the transaction size of primitives that impact the communication cost. The attribute size is 32 bytes, which corresponds to the output of SHA-256. Furthermore, the sizes of AuthConsent, CO.Prove, and DCP.Prove are 96 bytes, 480 bytes, and 1117 bytes, respectively.

8.2 Computation Cost

We evaluated the computation cost of each algorithm in the CO and DCP schemes. The execution time in milliseconds (μ) and the standard deviation (S.D) for a single attribute are presented in Table 3b and Table 3c, respectively. As shown in Table 3b, employing the BN-254 [5] curve, in CO scheme, outsourcing proof of one attribute takes 7.45 milliseconds, while verification requires 16.07 milliseconds. Based on Table 3c, in DCP scheme, delegable proof

Table 2: Communication Overhead (U:User, B:Blockchain, V:Verifier)

(a) CO Scheme

Process	U → B	B → U
KeyGen	$2\|g\|$	$2\|g\|$
ClientReg	$1\|g\| + 1\|PK\|$	$2\|g\| + 2\|PK\|$
RKGen	$1\|g\|$	n/a
AKGen	$q\|g\|$	n/a
CO.Prove	$2\|g\| + 1\|S\| + q\|PK\|$	n/a
CO.Verify	n/a	$\mathcal{O}(1)$

(b) DCP Scheme

Flow	AuthConsent	DCP.Prove	DCP.Verify
U → B	$1\|g\|$	n/a	n/a
U → V	$q\|g\| + \|req\| + \|ID\|$	n/a	n/a
B → U	n/a	n/a	n/a
B → V	n/a	$7\|g\| + 1\|S\| + 5\|PK\|$	n/a
V → U	$1\|g\|$	n/a	$\mathcal{O}(1)$
V → B	$1\|g\|$	n/a	n/a

Table 3: Experimental Results on Transaction Size and Computation Cost

(a) Transaction Size (TS)

Primitives	TS [Byte]
Attribute	32
Credential	132
AuthConsent	96
CO.Prove	480
DCP.Prove	1117

(b) CO Scheme Execution Time

Operation	μ [ms]		S.D. [ms]
	BN-254	BLS12-381	
KeyGen	3.19	4.80	0.42
ClientReg	14.30	21.45	0.91
RKGen	0.60	0.92	0.11
AKGen	1.1	0.31	1.60
CO.Prove	7.45	11.15	1.04
CO.Verify	16.07	24.10	1.76

(c) DCP Scheme Execution Time

Operation	μ [ms]		S.D. [ms]
	BN-254	BLS12-381	
AuthCommit	17.15	24.01	0.91
AuthConsent	2.04	3.1	0.31
DCP.Prove	35.10	50.85	1.39
DCP.Verify	51.42	72.10	1.93

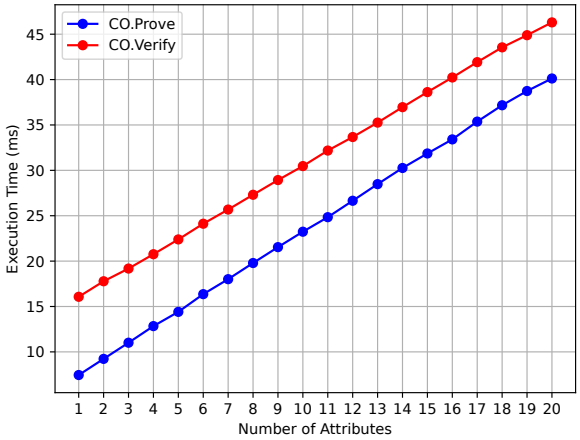

Figure 3: Credential Outsourcing Scheme

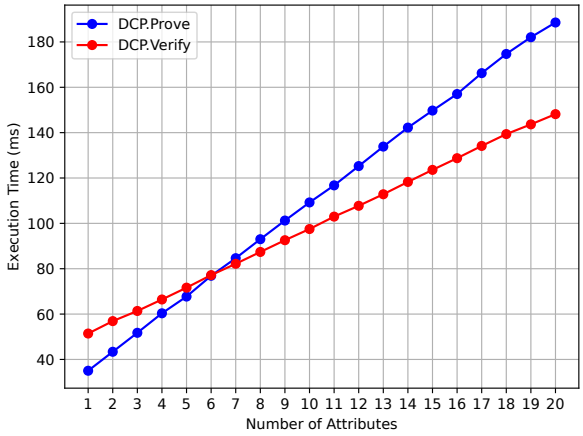

Figure 4: Delegable Credential Presentation Scheme

of one attribute takes 35.10 milliseconds, and verifying a delegable proof of a credential with one attribute requires 51.42 milliseconds, demonstrating efficiency. Figure 3 illustrates the execution time of CO.Prove and CO.Verify in the CO scheme, while Figure 4 shows the execution time of DCP.Prove and DCP.Verify in DCP scheme, considering a total of 20 attributes to prove in a credential. We limited the experiment to 20 attributes since proving knowledge of that many attributes in one user request is considered quite sufficient. The results indicate that the computation cost grows linearly with the number of attributes disclosed, but the execution time remains manageable in milliseconds.

9 CONCLUSION

We propose SilentProof, a blockchain-backed anonymous authentication protocol, to address the challenges of offering anonymous credentials as-a-service through a third party. Existing schemes are hindered by the structure of the credential presentation, making it difficult to efficiently provide anonymous credentials at scale. Moreover, resource-constrained devices face challenges in the computation-heavy credential presentation process. To overcome these limitations, SilentProof integrates blockchain technology and introduces a zero-knowledge proxy re-identification scheme. Outsourcing storage and computation to the blockchain enables seamless and efficient anonymous credential presentation while maintaining user privacy and control. We provide comprehensive proof of SilentProof's effectiveness and implement a functional prototype to demonstrate its efficiency and robustness. Our study fills a critical gap in the literature and offers a solution with a significant potential impact on privacy-preserving identity management systems, particularly for resource-constrained devices, and paves the way for potential modern use cases of anonymous credentials.

REFERENCES

[1] Erdem Aktas, Cfir Cohen, Josh Eads, James Forshaw, and Felix Wilhelm. 2023. *Intel Trust Domain Extensions (TDX) Security Review.* Technical Report. Google technical report.

[2] Ghada Arfaoui, Jean-François Lalande, Jacques Traoré, Nicolas Desmoulins, Pascal Berthomé, and Saïd Gharout. 2015. A practical set-membership proof for privacy-preserving NFC mobile ticketing. *Proceedings on Privacy Enhancing Technologies* 2015, 2 (2015), 25–45.

[3] Man Ho Au, Willy Susilo, Yi Mu, and Sherman SM Chow. 2012. Constant-size dynamic k-times anonymous authentication. *IEEE Systems Journal* 7, 2 (2012), 249–261.

[4] Amira Barki, Solenn Brunet, Nicolas Desmoulins, and Jacques Traoré. 2017. Improved algebraic MACs and practical keyed-verification anonymous credentials. In *The 23rd International Conference on Selected Areas in Cryptography–SAC 2016.* Springer, 360–380.

[5] Paulo SLM Barreto and Michael Naehrig. 2006. Pairing-friendly elliptic curves of prime order. In *Selected Areas in Cryptography: 12th International Workshop.* Springer, 319–331.

[6] David Bernhard, Olivier Pereira, and Bogdan Warinschi. 2012. How not to prove yourself: Pitfalls of the fiat-shamir heuristic and applications to helios. In *Advances in Cryptology–ASIACRYPT 2012.* Springer, 626–643.

[7] Dan Boneh and Xavier Boyen. 2004. Short signatures without random oracles. In *Advances in Cryptology-EUROCRYPT 2004.* Springer, 56–73.

[8] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short signatures from the Weil pairing. In *International conference on the theory and application of cryptology and information security.* Springer, 514–532.

[9] Sean Bowe. 2017. BLS12-381: New zk-SNARK elliptic curve construction. *Zcash Company blog, URL: https://z. cash/blog/new-snark-curve* 22 (2017).

[10] Stefan Brands. 2000. *Rethinking public key infrastructures and digital certificates: building in privacy.* Mit Press.

[11] Xavier Bultel and Pascal Lafourcade. 2017. Zero-knowledge proxy re-identification revisited. *Cryptology ePrint Archive* (2017).

[12] Jan Camenisch, Liqun Chen, Manu Drijvers, Anja Lehmann, David Novick, and Rainer Urian. 2017. One TPM to bind them all: Fixing TPM 2.0 for provably secure anonymous attestation. In *2017 IEEE Symposium on Security and Privacy (SP).* IEEE, 901–920.

[13] Jan Camenisch and Thomas Groß. 2012. Efficient attributes for anonymous credentials. *ACM Transactions on Information and System Security (TISSEC)* 15, 1 (2012), 1–30.

[14] Jan Camenisch and Anna Lysyanskaya. 2004. Signature schemes and anonymous credentials from bilinear maps. In *Advances in Cryptology–CRYPTO 2004.* Springer, 56–72.

[15] Jan Camenisch and Els Van Herreweghen. 2002. Design and implementation of the idemix anonymous credential system. In *Proceedings of the 9th ACM Conference on Computer and Communications Security.* 21–30.

[16] Melissa Chase, Chaya Ganesh, and Payman Mohassel. 2016. Efficient zero-knowledge proof of algebraic and non-algebraic statements with applications to privacy preserving credentials. In *Advances in Cryptology–CRYPTO 2016.* Springer, 499–530.

[17] Melissa Chase, Sarah Meiklejohn, and Greg Zaverucha. 2014. Algebraic MACs and keyed-verification anonymous credentials. In *Proceedings of the 2014 acm sigsac conference on computer and communications security.* 1205–1216.

[18] David L Chaum. 1981. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM* 24, 2 (1981), 84–90.

[19] Guoxing Chen, Yinqian Zhang, and Ten-Hwang Lai. 2019. Opera: Open remote attestation for intel's secure enclaves. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security.* 2317–2331.

[20] Liqun Chen and Rainer Urian. 2015. DAA-A: Direct anonymous attestation with attributes. In *The 8th International Conference on Trust and Trustworthy Computing.* Springer, 228–245.

[21] Victor Costan and Srinivas Devadas. 2016. Intel SGX explained. *Cryptology ePrint Archive* (2016).

[22] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. 2018. Privacy Pass: Bypassing Internet Challenges Anonymously. *Proc. Priv. Enhancing Technol.* 2018, 3 (2018), 164–180.

[23] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques.* Springer, 186–194.

[24] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. 2019. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology* 32 (2019), 498–546.

[25] Steven Galbraith, Florian Hess, and Frederik Vercauteren. 2008. Aspects of pairing inversion. *IEEE Transactions on Information Theory* 54, 12 (2008), 5719–5728.

[26] Christina Garman, Matthew Green, and Ian Miers. 2013. Decentralized anonymous credentials. *Cryptology ePrint Archive* (2013).

[27] Shafi Goldwasser, Silvio Micali, and Chales Rackoff. 2019. The knowledge complexity of interactive proof-systems. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali.* 203–225.

[28] Matthew Green and Ian Miers. 2017. Bolt: Anonymous payment channels for decentralized currencies. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security.* 473–489.

[29] Ulrich Haböck and Stephan Krenn. 2019. Breaking and fixing anonymous credentials for the cloud. In *International Conference on Cryptology and Network Security.* Springer, 249–269.

[30] Lucjan Hanzlik and Daniel Slamanig. 2021. With a little help from my friends: Constructing practical anonymous credentials. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security.* 2004–2023.

[31] Stephan Krenn, Thomas Lorünser, Anja Salzer, and Christoph Striecks. 2017. Towards attribute-based credentials in the cloud. In *International Conference on Cryptology and Network Security.* Springer, 179–202.

[32] Michael Z Lee, Alan M Dunn, Brent Waters, Emmett Witchel, and Jonathan Katz. 2013. Anon-pass: Practical anonymous subscriptions. In *2013 IEEE symposium on security and privacy.* IEEE, 319–333.

[33] Anna Lysyanskaya, Ronald L Rivest, Amit Sahai, and Stefan Wolf. 2000. Pseudonym systems. In *the 6th Annual International Workshop on Selected Areas in Cryptography, SAC'99.* Springer, 184–199.

[34] Milica Milutinovic, Koen Decroix, Vincent Naessens, and Bart De Decker. 2015. Privacy-preserving public transport ticketing system. In *29th Annual IFIP WG 11.3 Working Conf. on Data and Applications Security and Privacy.* Springer, 135–150.

[35] Christian Paquin and Greg Zaverucha. 2011. U-prove cryptographic specification v1. 1. *Technical Report, Microsoft Corporation* (2011).

[36] Christian Paquin and Greg Zaverucha. 2013. U-prove cryptographic specification v1. 1 (revision 3). *Released under the Open Specification Promise* (2013).

[37] David Pointcheval and Olivier Sanders. 2016. Short randomizable signatures. In *Cryptographers' Track at the RSA Conference.* Springer, 111–126.

[38] Kai Rannenberg, Jan Camenisch, and Ahmad Sabouri. 2015. Attribute-based credentials for trust. *Identity in the Information Society, Springer* (2015).

[39] Sietse Ringers, Eric Verheul, and Jaap-Henk Hoepman. 2017. An efficient self-blindable attribute-based credential scheme. In *The 21st International Conference on Financial Cryptography and Data Security (FC 2017).* Springer, 3–20.

[40] Claus-Peter Schnorr. 1989. Efficient identification and signatures for smart cards. In *Conference on the Theory and Application of Cryptology.* Springer, 239–252.

[41] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2018. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. *arXiv preprint arXiv:1802.07344* (2018).

A BLIND PS SIGNATURE

We utilize the blind PS signature for securing commitments and private attributes, ensuring efficient signing without the signer gaining knowledge of the underlying messages.

B.Setup$(1^\lambda) \rightarrow (params)$: The B.Setup algorithm on input of security parameter λ, outputs $params = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, g_1, g_2, e)$, where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$, are cyclic groups of order q that admit a type-3 pairing e.

B.KeyGen$(params) \rightarrow (sk, pk)$: The B.KeyGen algorithm selects $g_1 \xleftarrow{\$} \mathbb{G}_1^*, g_2 \xleftarrow{R} \mathbb{G}_2^*$ and $(x, y) \xleftarrow{R} \mathbb{Z}_p^2$, and computes $(X_1, Y_1) \leftarrow (g_1^x, g_1^y), (X, Y) \leftarrow (g_2^x, g_2^y)$, and sets $sk = X_1$, and $pk = (g_1, Y_1, g_2, \alpha, \beta)$.

B.BlindSign $(m) \rightarrow (\sigma)$: This protocol allows a user to obtain a signature on a message $m \in \mathbb{Z}_q$ without revealing any information about the message to the signer. The protocol is as follows:

(1) *Prepare*: On input a message m the user selects $t \xleftarrow{R} \mathbb{Z}_q$ and computes a Pedersen commitment $C \leftarrow g_1^t Y_1^m$. The user then sends C to the signer, along with a proof of knowledge of the opening of C as, $\Pi_s = PK\{(m, t) : C = g^t Y_1^m \wedge \phi_1(m) = 1\}$.
(2) *BlindSign*: Signer verifies the proof Π_s, and if accepts the proof, then selects $u \xleftarrow{R} \mathbb{Z}_q$, and computes a blind signature $\sigma' = (g_1^u, (XC)^u)$, and sends σ' to the requesting user.
(3) $Unblind(\sigma', t) \rightarrow \sigma$: The user takes the received blinded signature σ' and use the blinded factor t, parse the σ' as $\sigma' = (\sigma_1', \sigma_2')$ and computes a signature $\sigma \leftarrow (\sigma_1', \sigma_2'/(\sigma_1')^t)$.

The signature σ satisfies $\sigma_1 = g_1^u$, $\sigma_2 = (X_1C)^u/g_1^{ut} = (Xg_1^tY_1^m/g_1^t)^u = (X_1Y_1^m)^u$, which is a valid signature on m and serves as credential on attribute m.

B.Verify$(params, pk, m, \sigma) \rightarrow (true/false)$: The verification algorithm is the same as the main PS.Verify algorithm (Section 4.1). It parses the unblinded signature $\sigma = (\sigma_1, \sigma_2)$ and $pk = (g_2, \alpha, \beta)$. Output true if $\sigma_1 \neq 1_{\mathbb{G}_1}$ and $e(\sigma_1, XY^m) = e(\sigma_2, g_2)$, otherwise outputs false.

B SCHNORR'S PROTOCOLS

B.1 Protocol $\mathcal{I}_1$

Schnorr ZK Protocol for Identification:

- **Key Generation**: Choose a private key $sk \in \mathbb{Z}_p^*$ and compute the public key $pk = g_1^{sk}$.
- **Commitment**: The prover picks a random $a \in \mathbb{Z}_p^*$ and sends $A = g_1^a$ to the verifier.
- **Challenge**: The verifier sends a random challenge $c \in \mathbb{Z}_p^*$ to the prover.
- **Response**: The prover computes $\alpha = a + sk \cdot c$ and sends it to the verifier.
- **Verification**: The verifier checks if $g_1^\alpha = A \cdot pk^c$.

If the verification equation holds, verifier accepts the proof.

B.2 Protocol $\mathcal{I}_2$

Schnorr ZK Protocol in Bilinear Setting:

- **Key Generation**: Choose a private key $w_2 \in \mathbb{Z}_p^*$ and compute the public key $pk = e(g_1, g_2^{w_2})$.
- **Commitment**: The prover picks a random $b \in \mathbb{Z}_p^*$ and sends $B = e(g_1, g_2)^b$ to the verifier.
- **Challenge**: The verifier sends a random challenge $c \in \mathbb{Z}_p^*$ to the prover.
- **Response**: The prover computes $\beta = g_2^b \cdot w_2^c$ and sends it to the verifier.
- **Verification**: The verifier checks if $e(g_1, \beta) = B \cdot pk^c$.

If the verification equation holds, verifier accepts the proof.

C CORRECTNESS PROOF

This section presents correctness proof for DCP.Verify. Equation (4) is straightforward, and we omit the correctness proof for brevity.

C.1 Correctness Proof for equation (2):

$$e(h_1 \cdot g_1, \theta) = S \cdot e(h_1 \cdot g_1, pk_b)^d \cdot e\big(g_1, k\big)^{(d \cdot t_1)} \cdot T$$

From the left-hand side (LHS) of the equation (2):

$$\begin{aligned} LHS &= e\left(h_1 \cdot g_1, g_2^s \cdot (rk \cdot tk \cdot k_{att})^P\right) \\ &= e\left(h_1 \cdot g_1, g_2^s \cdot (g_2^{w/\alpha} \cdot k^{t_1/\alpha})^{m+r+\alpha \cdot d}\right) \\ &= e\Big(h_1 \cdot g_1, g_2\Big)^s \cdot e\Big(h_1 \cdot g_1, g_2\Big)^{(w/\alpha)\cdot(m+r)} \cdot e\Big(h_1 \cdot g_1, g_2\Big)^{(w \cdot d)} \\ &\quad \cdot e\Big(h_1 \cdot g_1, k\Big)^{(t_1/\alpha)\cdot(m+r)} \cdot e\Big(h_1 \cdot g_1, k\Big)^{(t_1 \cdot d)} \end{aligned}$$

From the right-hand side (RHS) of the equation (2):

$$\begin{aligned} RHS &= e\Big(h_1^m \cdot g_1^r, g_2^{w/\alpha}\Big) \cdot e\Big(h_1 \cdot g_1, g_2\Big)^s \\ &\quad \cdot e\Big(h_1 \cdot g_1, g_2^w\Big)^d \cdot e\Big(g_1, k\Big)^{(d \cdot t_1)} \cdot e\Big(h_1^m \cdot g_1^r, k^{t_1/\alpha}\Big) \\ &= e\Big(h_1 \cdot g_1, g_2\Big)^s \cdot e\Big(h_1 \cdot g_1, g_2\Big)^{(w/\alpha)\cdot(m+r)} \\ &\quad \cdot e\Big(h_1 \cdot g_1, g_2\Big)^{(w \cdot d)} \cdot e\Big(h_1 \cdot g_1, k\Big)^{(t_1/\alpha)\cdot(m+r)} \cdot e\Big(h_1 \cdot g_1, k\Big)^{(t_1 \cdot d)} \end{aligned}$$

Hence, LHS equals RHS in equation (2).

C.2 Correctness Proof for equation (3):

$$e(M \cdot R, x) \cdot e(h_1 \cdot g_1, y) = A \cdot B \cdot S^{t_2}$$

From the left-hand side (LHS) of the equation (3):

$$LHS = e\Big(h_1^m \cdot g_1^r, g_2^a \cdot g_2^{(w/\alpha)\cdot t_2}\Big) \cdot e\Big(h_1 \cdot g_1, g_2^{b+s \cdot t_2}\Big) =$$

$$e\Big(h_1^m \cdot g_1^r, g_2^a\Big) \cdot e\Big(h_1^m \cdot g_1^r, g_2^{(w/\alpha)}\Big)^{t_2} \cdot e\Big(h_1 \cdot g_1, g_2\Big)^b \cdot e\Big(h_1 \cdot g_1, g_2\Big)^{s \cdot t_2}$$

From the right-hand side (RHS) of the equation (3):

$$\begin{aligned} RHS = A \cdot B \cdot S^{t_2} &= e\Big(h_1^m \cdot g_1^r, g_2^a\Big) \cdot e\Big(h_1 \cdot g_1, g_2\Big)^b \\ &\quad \cdot e\Big(h_1^m \cdot g_1^r, g_2^{w/\alpha}\Big)^{t_2} \cdot e\Big(h_1 \cdot g_1, g_2\Big)^{s \cdot t_2} \end{aligned}$$

Hence, LHS equals RHS in equation (3).

C.3 Correctness Proof for equation (5)

$$e(\sigma_1', \gamma_1 \cdot M \cdot N) = e(\sigma_2' \cdot N, g_2)$$

From the left-hand side (LHS) of the equation (5):

$$\begin{aligned} LHS &= e\Big(h^r, g_2^x \cdot h_1^m \cdot h_0^t\Big) = e\Big(h^r, g_2^x \cdot e\big(\sigma_1', \beta\big)^m \cdot e\big(\sigma_1', g_2\big)^t\Big) \\ &= e\Big(h^r, g_2^x \cdot e\big(h^r, g_2^y\big)^m \cdot e\big(h^r, g_2\big)^t\Big) = e\big(h, g_2\big)^{(x+y \cdot m+t)\cdot r} \end{aligned}$$

From the right-hand side (RHS) of the equation (5):

$$RHS = e\Big(h^{(x+y \cdot m)\cdot r} \cdot e\big(h^r, g_2\big)^t, g_2\Big) = e\big(h, g_2\big)^{(x+y \cdot m+t)\cdot r}$$

Hence, LHS equals RHS in equation (5).

D MORE ON SECURITY PROOFS

D.1 Cryptographic Assumptions and Definitions

Definition 3 (Discrete Logarithm (DL)). *Let p be a prime number generated according to a security parameter $1^\lambda \in \mathbb{N}$. Let $\mathbb{G}$ be a multiplicative group of prime order p, and $g \in \mathbb{G}$ be a generator. Given an instance $y \in \mathbb{G}$ where $x \xleftarrow{R} \mathbb{Z}_p^*$ and $y = g^x$, the discrete logarithm assumption (DL) in $(\mathbb{G}, p, g)$ is to compute x. The DL assumption states that no* PPT *algorithm exists $\mathcal{A}$ that solves DL with non-negligible probability.*

Definition 4 (FAPI2 [25]). *Let $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ be three groups of prime order p, $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$ be two group elements, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a non-degenerate bilinear pairing. Given an instance $y \in \mathbb{G}_T$ where $\mathcal{D} \xleftarrow{\$} \mathbb{G}_2$ and $e(g_1, \mathcal{D}) = y$, the fixed argument pairing inversion 2 problem (FAPI2) in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$ is*

to compute $\mathcal{D}$. The FAPI2 assumption states that no PPT *algorithm exists that solves FAPI2 with non-negligible probability. That is, there exists a negligible function ϵ such that for all $\mathcal{A} \in \mathsf{PPT}(\lambda)$:*

$$\Pr\left[\mathcal{D} \xleftarrow{R} \mathbb{G}_2; \mathcal{D}' \leftarrow \mathcal{A}(e(g_1, \mathcal{D})) : \mathcal{D} = \mathcal{D}'\right] = \epsilon(\lambda)$$

Definition 5 (BDLV [11]). *Let $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ be three groups of prime order p, $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$ be two group elements, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a type-2 non-degenerate bilinear pairing. Given an instance (y_1, y_2) where $x \xleftarrow{\$} \mathbb{Z}_p^*$, $y_1 = g_1^x$ and $y_2 = g_1^{1/x}$, the bilinear discrete logarithm variant (BDLV) problem in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$ is to compute x. That is, the BDLV in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$ states that there exists a negligible function ϵ such that for all $\mathcal{A} \in \mathsf{PPT}(\lambda)$:*

$$\Pr\left[x' \xleftarrow{R} \mathbb{Z}_p^*; x' \leftarrow \mathcal{A}(g^x, g^{1/x}) : x = x'\right] \leq \epsilon(\lambda)$$

Theorem 4. *The Bilinear Discrete Logarithm Variant (BDLV) assumption in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$ is hard to solve under the DL assumption in $(\mathbb{G}_2, g_2, p)$.*

Proof. The [11] proved that BDLV is hard to solve under the DL assumption and with the hypothesis that e is type-2 paring. The *proof-idea* is as follows:

Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a type-2 non-degenerate bilinear pairing where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are three groups of prime order p and $g_2 \in \mathbb{G}_2$ be a generator. Suppose there exists a PPT adversary $\mathcal{A}$ that solves the BDLV assumption in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$ for g_1 chosen in the uniform distribution on G_1. Then, we build a PPT adversary $\mathcal{B}$ that solves the DL assumption in $(\mathbb{G}_2, g_2, p)$.The adversary $\mathcal{B}$ receives $h_2 = g_2^\alpha$ as an input and since e is a type-2 pairing, there exists a computationally efficient morphism $\psi : \mathbb{G}_2 \to \mathbb{G}_1$. The adversary $\mathcal{B}$ chooses $r \xleftarrow{R} \mathbb{Z}_p^*$ and computes $h_1 = \psi(g_1^r)$ and $g_1 = \psi(h_2^r) = \psi(g_2^r)^\alpha = h_1^\alpha$. Then we can write $h_1 = g_1^{1/\alpha}$ and it implies that (h_1, h_2) is an instance of BDLV assumption. Let $\beta = 1/\alpha$ as a solution then $\mathcal{B}$ runs $\beta' \leftarrow \mathcal{A}(h_1, h_2)$ and returns $\beta' = 1/\alpha'$, and we can see that $\beta' = \beta$ which implies that $1/\beta = \alpha$ which is the discrete log. That concludes that $\mathcal{B}$ breaks the DL assumption in $(\mathbb{G}_2, g_2, p)$ in polynomial time with same probability that $\mathcal{A}$ breaks BDLV in $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$. □

Definition 6 (Zero-knowledge Identification (ZKI) [27]). *A zero-knowledge identification (ZKI) scheme is a tuple of three algorithms $(Setup, KeyGen, Proof)$, where $Set(\lambda)$ returns a setup value η, the $KeyGen$ returns a pair of public and private keys $(\mathsf{pk}, \mathsf{sk})$, and $Proof$ is a two-party protocol between two* PPT *algorithms the prover P and the verifier V. At the end of the protocol, V returns a bit b. Such ZKI must satisfies following properties for any $\eta \leftarrow Set(\lambda)$ and $(\mathsf{pk}, \mathsf{sk}) \leftarrow KGen(\eta)$:* ***Completeness****. If P knows* sk*, he is able to convince V and V outputs 1 as:*

$$\Pr[b \leftarrow out_V(Proof\langle P(\mathsf{sk}); V(\mathsf{pk})\rangle : b = 1] = 1$$

Soundness*. If a dishonest prover P^* does not know* sk*, then P^* is not able to convince V except with negligible probability:*

$$\Pr\left[b \leftarrow out_V(Proof\langle P^*(\mathsf{pk}); V(\mathsf{pk})\rangle : b = 1\right] \leq \epsilon(\lambda)$$

Zero-knowledge*. During the protocol, a dishonest verifier V^* reveals nothing about the* sk *except* pk*. That is for any possible dishonest verifier V^* there exists a* PPT *algorithm Sim called simulator, such that for any bit-string $\delta \in \{0,1\}^*$, the following equation holds:*

$$\Pr\left[\delta' \leftarrow out_{V^*}(Proof\langle P(\mathsf{sk}); V^*(\mathsf{pk})\rangle : \delta = \delta'\right]$$
$$= \Pr\left[\delta' \leftarrow Sim(\mathsf{pk}) : \delta = \delta'\right]$$

D.2 The CO Scheme Security Proof

The probability distribution for the knowledge soundness of Theorem (2) is as follows: By hypothesis, we know that:

$$\Pr[h_1^{z_{m,0}} = M' \cdot M^{c_0}] \geq \epsilon(\lambda), \quad \Pr[h_1^{z_{m,1}} = M' \cdot M^{c_1}] \geq \epsilon(\lambda)$$
$$\Pr[g_1^{z_{r,0}} = R' \cdot R^{f_0}] \geq \epsilon(\lambda), \quad \Pr[g_1^{z_{r,1}} = R' \cdot R^{f_1}] \geq \epsilon(\lambda)$$
$$\Pr[g_1^{z_{\alpha,0}} = pk'_{u,1} \cdot pk_{u,1}^{d_0}] \geq \epsilon(\lambda), \quad \Pr[g_1^{z_{\alpha,1}} = pk'_{u,1} \cdot pk_{u,1}^{d_1}] \geq \epsilon(\lambda)$$
$$\Pr[k^{z_{\beta,0}} = k'_{att} \cdot k_{att}^{e_0}] \geq \epsilon(\lambda), \quad \Pr[k^{z_{\beta,1}} = k'_{att} \cdot k_{att}^{e_1}] \geq \epsilon(\lambda)$$

and $(c_0, c_1, d_0, d_1, e_0, e_1, f_0, f_1)$ are chosen at random in $\mathbb{Z}_p^*$, we have $\Pr[c_0 = c_1] = 1/(p-1)$, $\Pr[d_0 = d_1] = 1/(p-1)$, $\Pr[e_0 = e_1] = 1/(p-1)$, and $\Pr[f_0 = f_1] = 1/(p-1)$ which is negligible. Thus, we conclude that:

$$\Pr[m \leftarrow \mathcal{K}^{P^*(bit)}] \geq \Pr[h_1^{z_{m,0}} = M' \cdot M^{c_0}] \cdot \Pr[h_1^{z_{m,1}} = M' \cdot M^{c_1}]$$
$$- \Pr[c_0 = c_1] \geq \epsilon(\lambda)^2 - 1/(p-1)$$
$$\Pr[r \leftarrow \mathcal{K}^{P^*(bit)}] \geq \Pr[g_1^{z_{r,0}} = R' \cdot R^{f_0}] \cdot \Pr[g_1^{z_{r,1}} = R' \cdot R^{f_1}]$$
$$- \Pr[f_0 = f_1] \geq \epsilon(\lambda)^2 - 1/(p-1)$$
$$\Pr[\alpha \leftarrow \mathcal{K}^{P^*(bit)}] \geq \Pr[g_1^{z_{\alpha,0}} = pk'_{u,1} \cdot pk_{u,1}^{d_0}] \cdot \Pr[g_1^{z_{\alpha,1}} = pk'_{u,1} \cdot pk_{u,1}^{d_1}]$$
$$- \Pr[d_0 = d_1] \geq \epsilon(\lambda)^2 - 1/(p-1)$$
$$\Pr[\beta/\alpha \leftarrow \mathcal{K}^{P^*(bit)}] \geq \Pr[k^{z_{\beta,0}} = k'_{att} \cdot k_{att}^{e_0}] \cdot \Pr[k^{z_{\beta,1}} = k'_{att} \cdot k_{att}^{e_1}]$$
$$- \Pr[e_0 = e_1] \geq \epsilon(\lambda)^2 - 1/(p-1)$$

D.3 The DCP Scheme Security Proof

The probability distribution for knowledge soundness of Theorem (3) is as follows: Based on the hypothesis, we have the following:

$$P_1 = \Pr\left[\left(e(h_1 \cdot g_1, \theta_0) = S_0 \cdot e(h_1 \cdot g_1, pk_b)^d \cdot e(g_1, k)^{d \cdot t_{1,0}} \cdot T_0\right)\right.$$
$$\left. \wedge \left(e(M \cdot R, x_0) \cdot e(h_1 \cdot g_1, y_0) = A_0 \cdot B_0 \cdot S_0^{t_{2,0}}\right)\right]$$
$$P_2 = \Pr\left[\left(e(h_1 \cdot g_1, \theta_1) = S_1 \cdot e(h_1 \cdot g_1, pk_b)^d \cdot e(g_1, k)^{d \cdot t_{1,1}} \cdot T_1\right)\right.$$
$$\left. \wedge \left(e(M \cdot R, x_1) \cdot e(h_1 \cdot g_1, y_1) = A_1 \cdot B_1 \cdot S_1^{t_{2,1}}\right)\right]$$

Moreover, since $t_0 = (t_{1,0} + t_{2,0})$, $t_1 = (t_{1,1} + t_{2,1})$ are chosen at random in $\mathbb{Z}_p^*$, we have:

$$P_3 = \Pr[t_{1,0} = t_{2,0} \vee t_{1,0} = t_{2,1} \vee t_{2,0} = t_{1,1} \vee t_{2,0} = t_{1,1}]$$
$$\leq \Pr[t_{1,0} = t_{2,1}] + \Pr[t_{2,0} = t_{1,1}] + \Pr[t_{2,0} = t_{1,1}] \leq \frac{4}{p-1}$$

which is negligible. We conclude that for witness $w = (rk, k^{1/r}, \beta, P)$ and $com = (A, B, S)$:

$$\Pr[w \leftarrow \mathcal{K}^{P(\text{bit})}(com)] \geq P_1 \cdot P_2 - P_3 \geq \epsilon(\lambda)^4 - \frac{4}{p-1}$$

In equation (5), the t can be extracted as:

$$t = \frac{z_{t,1} - z_{t,0}}{c_{2,1} - c_{2,0}} = \frac{(t' + t \cdot c_{2,1}) - (t' + t \cdot c_{2,0})}{c_{2,1} - c_{2,0}} = \frac{t(c_{2,1} - c_{2,0})}{c_{2,1} - c_{2,0}}$$

D.4 Security Proof of SilentProof

Proof. Based on Theorem 1, we sketch the following security proof of SilentProof protocol.

Unforgeability: The PS signature scheme [37] or Coconut signature schemes [41], used for issuing user credentials, is proven to be existentially unforgeable under chosen message attacks. Given that the user later presents these credentials to the blockchain in the CO scheme and subsequently by the blockchain to a verifier in the DCP scheme, the unforgeability against any adversary forging a credential proof is maintained. That is, unforgeability is ensured by the soundness property of the zero-knowledge proof in both CO (Section D.2) and DCP (Section D.3) schemes.

Blindness: The PS and Coconut signature schemes permit users to receive a signature on a commitment rather than the actual value m. Both schemes affirm blindness under the XDH assumptions [8], ensuring no attribute m data is disclosed to the issuer during the credential issuance. The same credentials are later used by the user in the CO scheme and by the blockchain in the DCP scheme. Commitments in the CO scheme (M, R) based on DL assumption and those in DCP (S, A, B), based on BDLV assumption, confirm that neither the blockchain nor verifier gets any insight into attribute m. Crucially, the plaintext m remains undisclosed to both entities throughout the verification. Moreover, the zero-knowledge property ensures that neither CO.Prove nor DCP.Prove divulges the attribute value m or the secret keys sk_u, sk_b.

Unlinkability: Under the DL assumption, unlinkability is maintained. In the CO.Prove phase of the CO scheme; credentials are initially re-randomized, and again in DCP.Prove using factors (r, t). The delegated zero-knowledge proofs are randomized through factors (s, a, b) and further adjusted by verifier challenges $(t = t_0 + t_1)$ in DCP.Prove, ensuring unlinkability during each presentation.

Collusion: In the event of collusion between the blockchain and verifier, the extraction of the user secret key $(sk = \alpha)$ from the re-proof key $(rk = g_2^{w/\alpha})$ remains unattainable, as per the BDLV assumption shown in the proof found in (Section 4). Our rk is constructed based on the [11] scheme, and it shows in [11] that collusion between a proxy and verifier does not compromise the confidentiality of the user secret key. Similarly, a malicious user cannot extract the blockchain secret key, as only the blockchain's public key in rk is utilized, as proven in [11]. Furthermore, the attribute private key (β) cannot be extracted from the attribute verification key $(k_{att} = k^{\beta/\alpha})$, and the one-time attribute token $(tk = k^{(t_1-\beta)/\alpha})$ and remains secure under the DL assumption, and private value (β) is canceled out within the DCP.Prove algorithm.

Zero-knowledge: The zero-knowledge attributes of CO (Section D.2) and DCP (Section D.3) ensure that SilentProof is complete, sound, and zero-knowledge. Consequently, both CO.Prove and DCP.Prove only verifies the validity of a statement or possession of an attribute, selectively disclosing necessary public attributes without revealing excess information.

Security: SilentProof is resilient against following attacks:

Algorithm 1: Multi-Attribute CO System Initialization

Data: $params = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e, H, H_1)$

KeyGen

User selects $\alpha \xleftarrow{R} \mathbb{Z}_p^*$ and generates keys as:
$sk_u = \alpha, \quad pk_u = (pk_{u,1}, pk_{u,2}) = (g_1^{\alpha}, g_2^{1/\alpha})$
Blockchain selects $w \xleftarrow{R} \mathbb{Z}_p^*$ and generates keys as:
$sk_b = w, \quad pk_b = (pk_{b,1}, pk_{b,2}) = e(g_1, g_2^{w})$

ClientReg

User chooses $s_1 \xleftarrow{R} \mathbb{Z}_p^*$ and computes:
$v_1 = g_1^{s_1}, \quad \pi_{s_1} = PK\{s_1 : v_1 = g_1^{s_1}\}$
User sends (v_1, π_{s_1}) to the blockchain.
Blockchain verifies π_{s_1}, and if it holds, then computes:
Let $s = s_1 + s_2$ where $(s_2, o) \xleftarrow{R} \mathbb{Z}_p^*$
$v_1 \cdot g_1^{s_2} = g_1^{s_1+s_2} = v$
$\mu = (g_1^{s} h)^{1/(\mathrm{sk}_b \cdot o)}$
$\pi_{s_2} = \{(s, o) : \mu^{sk_b} = v \cdot h \cdot \mu^{-o}\}$
Choose a random pseudonymous identifier ID_u and sends $(ID_u, \mu, g_1^{s_2}, \pi_{s_2})$ to the user and save (μ, s_2, v)

RKGen

User computes re-proof key rk, sends it to blockchain:
$rk = (pk_{b,2})^{\alpha} = (g_2^{w})^{1/\alpha} = g_2^{w/\alpha}$

AKGen

For each attribute i, user generates attribute verification key k_{att_i} and attribute private key β_i as:
for $i \leftarrow 1$ *to* q **do**
$\beta_i = (\tau_i + \mu \cdot s_1)$ where $\tau_i \xleftarrow{R} \mathbb{Z}_p^*$
$k_i = H_1(att_i), \quad k_{att_i} = \prod_{i=1}^{q} k_i^{\beta_i/\alpha}$

(1) *Impersonation Attack*: The user's consent is essential for the blockchain to create proof on their behalf. Without the user-consent token $tk = k^{\alpha/\beta}$, which necessitates both the user's secret key α and the attribute key β, a malicious blockchain cannot impersonate the user or disseminate their credentials. For unauthorized access, a malicious user needs the victim user's secret key α, registration secrets (μ, τ, s_1) and each attribute's private key β to generate tk.
(2) *Replay Attack*: Each verification requires a one-time user's tk and a fresh challenge $t = t_0 - t_1$ from the verifier to both the blockchain and user, preventing from reusing prior proofs.
(3) *Sybil Attack*: Users must register with the blockchain and obtain a delegation permission token μ. This token later binds to the user-consent token, thwarting unregistered malicious users from executing a Sybil attack, as they would need a legitimate user secret key, attribute key, and the blockchain's delegation permission token.

This concludes that SilentProof protocol upholds anonymous credentials through its unforgeability, blindness, multi-show unlinkability, and collusion. It retains zero-knowledge properties, ensuring

Algorithm 3: Multi-Attribute DCP Scheme

Data: $pp = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e, H_1, H)$, (h_0, h_i) for $i \in [1, \ldots, q]$

AuthCommit

Blockchain selects $(s, a, b) \xleftarrow{R} Z_p^*$ and computes commitments (S, A, B) as:

$$S = \prod_{i=1}^{q} e(M_i \cdot R, rk) \cdot \prod_{i=1}^{q} e(h_i \cdot g_1, g_2)^s,$$

$$A = \prod_{i=1}^{q} e(M_i \cdot R, g_2^a), \quad B = \prod_{i=1}^{q} e(h_i \cdot g_1, g_2)^b$$

AuthConsent

The user sends $(service, ID_u, att_i)$ as a QR code to the verifier.

Verifier selects $t = t_0 + t_1 \xleftarrow{R} \mathbb{Z}_p^*$, sends t_1 to user, and (ID_u, att_i, t_0) to blockchain.

User generates one-time attribute token as: $tk_i = \prod_{i=1}^{q} (k_i^{(t_1-\beta_i)/\alpha})$

User sends tk_i to blockchain to authorize attribute disclose proof.

DCP.Prove

Blockchain computes a new challenge as $t_2 = H(t_0 || H(S, A, b))$

Blockchain computes response (x, y, θ, T) as:

$$x = g_2^a \cdot rk^{t_2}, \; y = g_2^{b+s \cdot t_2},$$

$$\theta = g_2^s \cdot \prod_{i=1}^{q} (rk \cdot tk_i \cdot k_{att_i})^{P_i}, \; T = \prod_{i=1}^{q} e(M_i \cdot R, tk_i \cdot k_{att_i})$$

Chooses $(r', t) \xleftarrow{R} \mathbb{Z}_p^*$, and randomize and blinded PS signature as: $\sigma' = (\sigma_1', \sigma_2') = (\sigma_1^{r'}, (\sigma_2 \cdot \sigma_1^t)^{r'})$

Prove blinding factor t by choosing t'

Computes: $N = {h_0}^t$, $N' = h_0^{t'}$, $c_2 = H(N, N')$, $z_t = t' + t \cdot c_2$

Blockchain sends $\Theta = ((S, A, B, N, N'), (x, y, \theta, T, z_t), (t_2, c_2))$ to service provider.

DCP.Verify

The Verifier checks

if $\prod_{i=1}^{q} e(h_i \cdot g_1, \theta) = S \cdot \prod_{i=1}^{q} e(h_i \cdot g_1, pk_b)^d \cdot e(g_1, k)^{(d \cdot t_1)} \cdot T$ *and* $\prod_{i=1}^{q} e(M_i \cdot R, x) \cdot \prod_{i=1}^{q} e(h_i \cdot g_1, y) = A \cdot B \cdot S^{t_2}$ *and* $h_0^{z_t} = N' \cdot N^{c_2}$ *and* $\sigma_1' \neq 1_{\mathbb{G}_1}$, $e(\sigma_1', \gamma_1 \cdot \prod_{i=1}^{q} M_i \cdot N) = e(\sigma_2' \cdot N, g_2)$ **then**

Return "Accept"

else

Return "Reject"

correctness, soundness, and honest-verifier zero-knowledge while also defending against impersonation, replay, and Sybil attacks. □

Algorithm 2: Multi-Attribute CO Verification

Data: $params = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e, H, H_1)$

CO.Prove

Runs PS.Prove algorithm for $(\sigma, (m_1, \ldots, m_q))$ as:

begin

Parse issuer public key as: $(g_2, X, Y_i) = (g_2, g_2^x, g_{2,i}^{y_i})$ for $i \in [1, \ldots, q]$

Parse signature as: $\sigma = (\sigma_1, \sigma_2) = (h, h^{x+y_i \cdot m_i})$

Selects $(r_0, t_o) \xleftarrow{R} \mathbb{Z}_p^*$, computes $\sigma' = (\sigma_1', \sigma_2') = (\sigma_1^{r_0}, (\sigma_2 \cdot \sigma_1^{t_0})^{r_0})$

Sends σ' to blockchain, carries zero-knowledge proof for $(t_0, m_1, \ldots, m_q)$ as:

$$\pi_{sign} = PK\{(m_1, \ldots m_q, t_0) : e(\sigma_1', X) \cdot \prod_{i=1}^{q} e(\sigma_1', Y_i)^{m_i} \cdot e(\sigma_1', g_2)^t = e(\sigma_2', g_2)\}$$

User generates a commitment M_i for each m_i and zero-knowledge proof π_{co} as:

Refactor equation (5) as:

$$com = e(\sigma_2', g_2)/e(\sigma_1', X) \in \mathbb{G}_T,$$

$$h_0 = e(\sigma_1', g_2), \quad h_i = \prod_{i=1}^{q} e(\sigma_1', Y_i)$$

Compute commitments $M_i = \prod_{i=1}^{q} h_i^{m_i}$, $R = g_1^r$, and proof π_{co} as:

$$\pi_{co} = PK\{(m_1, \ldots, m_q, r, \alpha, \beta_1, \ldots, \beta_q) : M_i = \prod_{i=1}^{q} h_i^{m_i} \wedge R = g_1^r \wedge pk_{u,1} = g_1^\alpha \wedge k_{att_i} = \prod_{i=1}^{q} k_i^{\beta_i/\alpha}\}$$

Computes $P_i = \prod_{i=1}^{q} m_i + r + \alpha \cdot d$

User sends $(\sigma', \pi_{sign}, \pi_{co}, M_i, P_i, \beta_i)$ to blockchain

CO.Verify

Blockchain verifies;

if π_{sign} *is valid through PS.Verify and* π_{co} *is valid then store* $(\sigma', M_i, P_i, \beta_i)$ *for* $i \in [1, \ldots, q]$ **then**

Return "Accept"

else

Return "Reject"

E MULTI-ATTRIBUTES SILENTPROOF

In this section, we present an extended version of SilentProof that supports multiple attributes for both CO and DCP schemes. The CO scheme is present in Algorithms (1) and (2), while the DCP scheme is presented in Algorithm (3) as follows.